SISTEMA DE GESTIÓN EMPRESARIAL

UD07. Desarrollo de módulos de Odoo Controlador, Herencia y Web Controllers

Actividades Evaluables 01



Miriam Mendieta Racedo.-
2DAMS
18/01/2026

ÍNDICE

INTRODUCCIÓN **1**

ACTIVIDAD 01 2
ACTIVIDAD 02 15
ACTIVIDAD 03 25
ACTIVIDAD 04 32

BIBLIOGRAFIA **38**

INTRODUCCIÓN

El presente proyecto tiene como objetivo profundizar en la personalización y extensión de capacidades del ERP Odoo 18, centrándose en la flexibilidad que ofrece el framework para adaptarse a necesidades de negocio específicas. A través de una serie de actividades prácticas, se implementarán soluciones que abarcan desde la lógica interna del servidor hasta la comunicación con servicios externos como Telegram.

La herencia es el pilar fundamental de Odoo. En este proyecto, se utiliza para:

- **Modularidad**: Permite modificar el comportamiento del módulo EJ07-LigaFutbol sin alterar su código original.
- **Mantenibilidad**: Al heredar clases, los cambios se mantienen aislados, lo que facilita las actualizaciones futuras del sistema.
- **Extensibilidad**: permite añadir nuevos campos (como los puntos especiales por diferencia de goles) y nuevos métodos (como la suma masiva de goles) de forma limpia.

El uso de **Web Controllers** (como el solicitado para la URL /eliminarempates) permite que Odoo actúe como un servicio web real. Sus beneficios incluyen:

- **Integración Externa**: Permite que aplicaciones de terceros o llamadas HTTP simples ejecuten acciones dentro de la base de datos de Odoo.
- **Automatización**: Facilita la creación de "hooks" para tareas de mantenimiento, como la eliminación de registros duplicados o empatados en este caso.
- **Flexibilidad de Salida**: Puede devolver desde texto plano hasta imágenes generadas dinámicamente con librerías como PIL.

Integración con APIs y Bots (Telegram)

La actividad de conectar un **Bot de Telegram** con una **API REST** demuestra la capacidad de Odoo para formar parte de un ecosistema digital más amplio. Esto permite a los usuarios interactuar con la gestión empresarial desde dispositivos móviles de forma inmediata, facilitando operaciones de consulta y edición de datos (socios) sin necesidad de entrar a la interfaz completa de Odoo.

Finalmente, la implementación de **vistas Graph** e **informes PDF** responde a la necesidad de transformar los datos brutos en información visual y documentos formales para la toma de decisiones

ACTIVIDAD 01

Modifica el ejemplo de “EJ07-LigaFútbol” de forma que:

- Los partidos por 4 o más goles de diferencia, computan como “4” puntos para el ganador y “-1” punto para el perdedor.
- Implementa a la vista de los partidos un botón que al pulsarlo, sume 2 goles a todos los equipos de casa en todos los partidos disputados. La clasificación deberá recalcularse.
- Implementa a la vista de los partidos un botón que al pulsarlo, sume 2 goles a todos los equipos visitantes en todos los partidos disputados. La clasificación deberá recalcularse.
- Implemente una llamada web mediante un Web Controller en la URL : http://localhost:8069/eliminarempates que cuando se haga, elimine todos los partidos que estén empatados. Mostrará el número de partidos eliminados.
- Genera un informe para cada partido que muestre el resultado del partido en PDF.
- Genera un Wizard para crear nuevos partidos.
- Implementa a la vista de los partidos una vista “Graph” que muestre de forma visual estadísticas de los goles marcados por partidos en casa.

Preparación previa

1. Transferir la capeta modulo EJ07-LigaFutbol desde anfitrión a VM y mover a addons:
   scp -r -P 2222 "/ruta/absoluta/a/ EJ07-LigaFutbol " miriam@127.0.0.1:~/
   mv ~/ EJ07-LigaFutbol ~/futbol/desarrollo/volumesOdoo/addons/

2. Navegar a la carpeta addons y otorgar permisos:
   sudo chmod -R 777 ./ EJ07-LigaFutbol

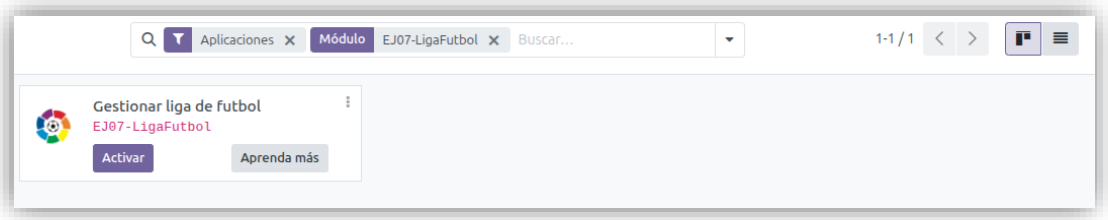
3. Desde la carpeta desarrollo reiniciar el contenedor :
   docker restart desarrollo-web-1

4. En la interfaz web de Odoo(http://localhost:8069): activar el Modo Desarrollador (en Ajustes) → menú Aplicaciones → Actualizar lista de aplicaciones → Buscar "LigaFutbol" e instalar.

5. En la views se modificó tree por list ya que daba error. Se reinicia el contendor para que reconozca los cambios.(*docker restart desarrollo-web-1*) y se vuelve a actualizar.

Estructura de Carpetas en el Servidor

```
futbol/                          # Carpeta Raíz del Proyecto
├── produccion/                  # (Vacía por ahora) Entorno para el cliente final
└── desarrollo/                  # entorno de trabajo actual
    ├── docker-compose.yml       # Configuración de los contenedores (Odoo y DB)
    └── volumesOdoo/             # Persistencia de datos de Odoo
    └── addons/                  # módulos nuevos para Odoo
        └── EJ07-LigaFutbol/     # módulo de Odoo
            ├── __init__.py      # Importa la carpeta 'models' y 'wizard'
            ├── __manifest__.py  # Metadatos del módulo y registro de archivos XML
            ├── controllers/     # (Lógica para rutas web/HTTP)
            │   └── __init__.py
            ├── models/          # Lógica del módulo (Python)
            │   ├── __init__.py  # Importa los archivos .py de esta carpeta
            │   ├── liga_equipo.py  # Define el modelo 'liga.equipo' (nombre, escudo...)
            │   └── liga_partido.py # Lógica de partidos y cálculo de puntos
            ├── report/          # Configuración de informes impresos
            │   └── liga_report.xml # Diseño de la plantilla PDF para la clasificación
            ├── security/        # Permisos de acceso
            │   └── ir.model.access.csv # Define quién puede leer/escribir en el módulo
            ├── static/          # Archivos visuales
            │   └── description/
            │   └── icon.png     # El icono que se verá en el menú de aplicaciones
            ├── views/           # Diseño de las pantallas (Interfaz)
            │   ├── liga_equipo.xml  # Vistas (list/form) y Menús para Equipos
            │   ├── liga_partido.xml # Vistas (list/form) y Menús para Partidos
            │   └── liga_equipo_clasificacion.xml # Vista especial para ver la tabla
            └── wizard/          # Asistentes temporales
                ├── __init__.py  # Importa el archivo .py del wizard
                ├── liga_equipo_wizard.py # Lógica del Wizard (TransientModel)
                └── liga_equipo_wizard_view.xml # Interfaz del Wizard (ventana emergente)
```



Lógica de Puntos Especiales (Herencia de Modelo)

Se crea liga_futbol_pro que tiene como dependencia a EJ07-LigaFutbol. Esto garantiza que la lógica de negocio se mantenga desacoplada y sea fácil de gestionar.

Se utiliza la herencia de extensión para no alterar el módulo original. El decorador @api.depends asegura que los puntos se actualicen automáticamente en tiempo real cada vez que los goles cambien, y store=True permite realizar estadísticas visuales sobre estos datos.




```
addons/
    |
    ├──── EJ07-LigaFutbol/          <-- Módulo original
    |
    └──── liga_futbol_pro/          <-- módulo herencia
            ├──── __init__.py              <-- Importa las carpetas: models, wizard, report, controllers
            ├──── __manifest__.py       <-- El manifiesto que conecta con el original
            ├──── models/
            |           ├──── __init__.py
            |           └──── liga_partido_inherit.py <-- código de herencia
            ├──── views/
            |           └──── liga_partido_inherit_views.xml
            └──── controllers/          <-- Carpeta principal de controladores
                        ├──── __init__.py       <--  Importa el archivo 'main.py'
                        └──── main.py           <--  Contiene la lógica de /eliminarempates
```

Dentro del módulo liga_futbol_pro se implementará una nueva regla de puntuación para los partidos en la liga:

Partidos con diferencia de 4 o más goles:

- El ganador del partido recibirá 4 puntos.
- El perdedor del partido recibirá -1 punto (un punto negativo).

Models/liga_partido_inherit.py

```
# -*- coding: utf-8 -*-
from odoo import models, fields, api

class LigaPartidoInherit(models.Model):
    """Se  hereda el modelo original de la liga de futbol para modificar
       La lógica de asignación de puntos según los resultados.
    """
    _inherit = 'liga.partido'

    # Campos computados para los puntos especiales
    puntos_casa = fields.Integer(string="Puntos Local", compute="_compute_puntos_actividad", store=True)
    puntos_visitante = fields.Integer(string="Puntos Visitante", compute="_compute_puntos_actividad", store=True)

    @api.depends('goles_casa', 'goles_visitante')
    def _compute_puntos_actividad(self):
        for record in self:
            # Calculamos la diferencia de goles
            diferencia = abs(record.goles_casa - record.goles_visitante)

            # Lógica solicitada: Diferencia >= 4 -> 4 pts ganador, -1 pt perdedor
            if diferencia >= 4:
                if record.goles_casa > record.goles_visitante:
                    record.puntos_casa = 4
                    record.puntos_visitante = -1
                else:
                    record.puntos_casa = -1
                    record.puntos_visitante = 4
            else:
                # Lógica estándar de fútbol (3-1-0)
                if record.goles_casa > record.goles_visitante:
                    record.puntos_casa = 3
                    record.puntos_visitante = 0
                elif record.goles_visitante > record.goles_casa:
                    record.puntos_casa = 0
                    record.puntos_visitante = 3
                else:
                    record.puntos_casa = 1
                    record.puntos_visitante = 1

    goles_casa_incrementados = fields.Boolean(string="Goles Casa Incrementados", default=False)
    goles_fuera_incrementados = fields.Boolean(string="Goles Fuera Incrementados", default=False)

    def action_masive_goles_casa(self):
        # Solo busca los que no tienen su campo específico en True
        partidos = self.search([('goles_casa_incrementados', '=', False)])

        if not partidos:
            raise UserError("Los goles de casa ya han sido incrementados anteriormente.")

        for p in partidos:
            p.goles_casa += 2
            p.goles_casa_incrementados = True
        return True

    def action_masive_goles_visitante(self):
        # Solo busca los que no tienen su campo específico en True
        partidos = self.search([('goles_fuera_incrementados', '=', False)])

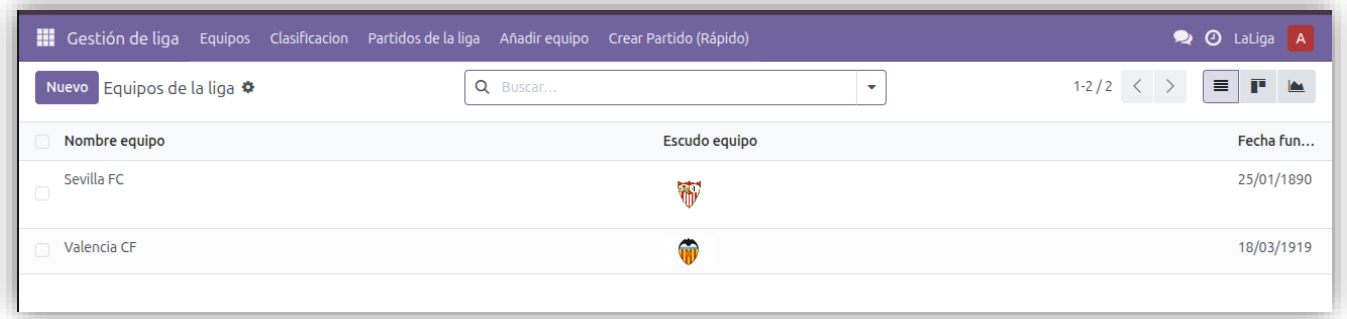
        if not partidos:
            raise UserError("Los goles de visitante ya han sido incrementados anteriormente.")

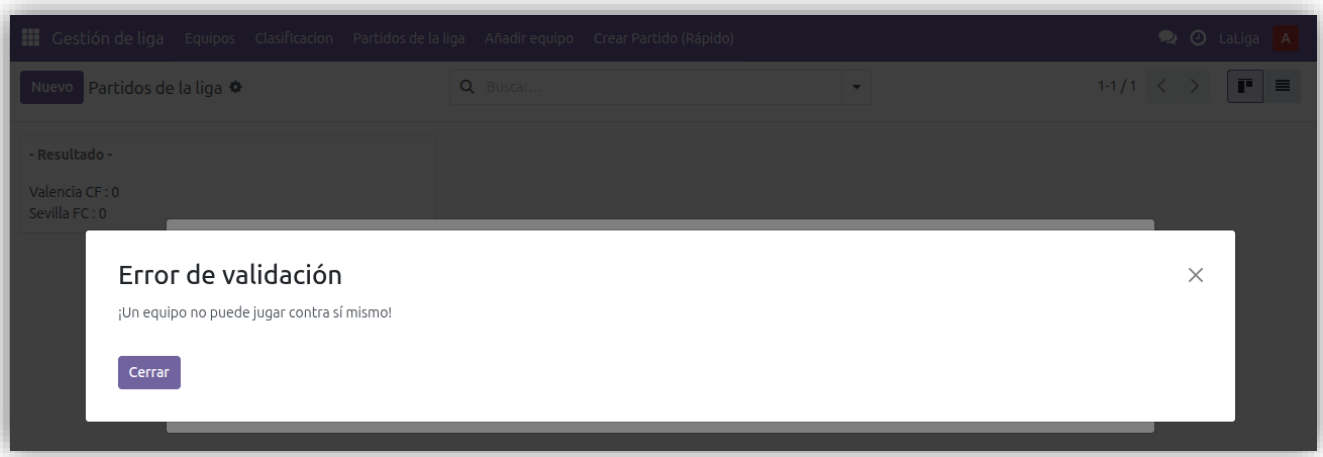
        for p in partidos:
            p.goles_fuera += 2
            p.goles_fuera_incrementados = True
        return True
```

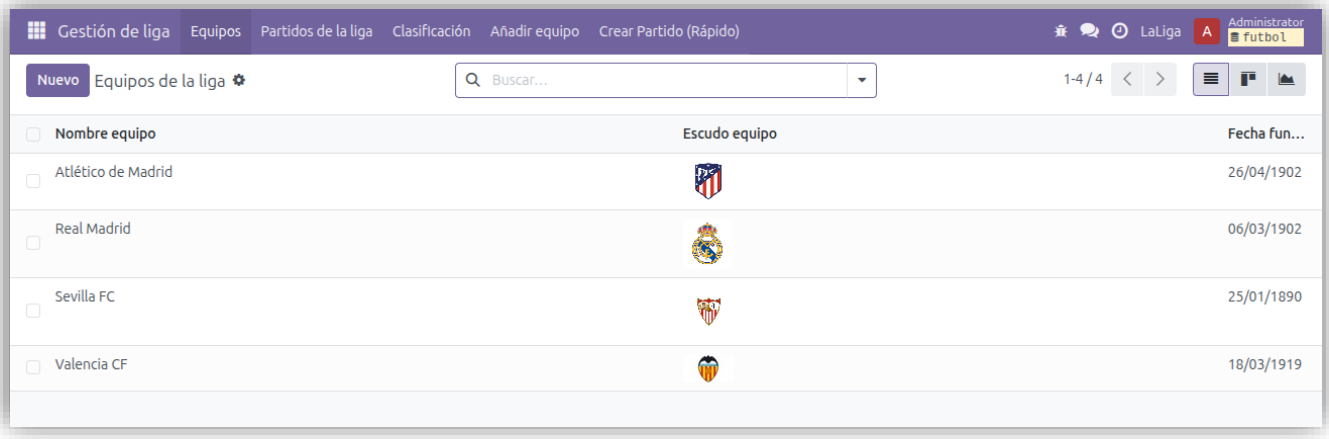
- En liga_futbol_pro /__init__.py: *from . import models*
- En liga_futbol_pro /models/__init__.py: *from . import liga_partido_inherit*

liga_futbol_pro / __manifest__.py

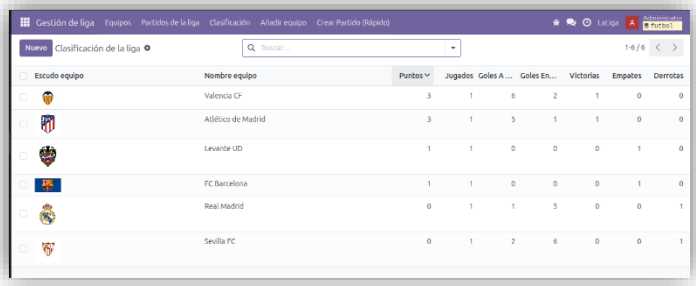
```
# -*- coding: utf-8 -*-
    {
    'name': 'Gestionar Liga Futbol PRO ',
     'summary: "Extesión avanzada para la Liga de futbol",
    'descripcion' :
            """
            Este módulo incluye mejoras mediante herencia para:
            - Puntuación especial por goleada
            -Eliminar partidos empatados
            -generar informes pdf de los partidos
            - creación de nuevos partidos mediante wizard
            """
   'author': "Miriam Mendieta",
    'category': 'Tools',
    'version': '1.0',
    'depends': ['base', 'EJ07-LigaFutbol'], # dependencias del módulo
    'data': [
        'security/ir.model.access.csv',
        'wizard/partido_wizard_view.xml',
        'views/liga_partido_inherit_views.xml',
        'report/partido_report.xml',
    ],
    'license': 'LGPL-3'
}
```

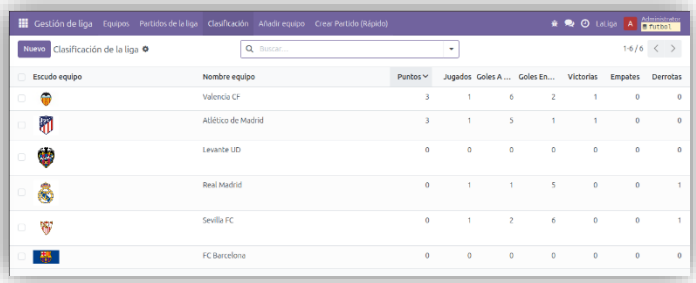






Eliminación de empate





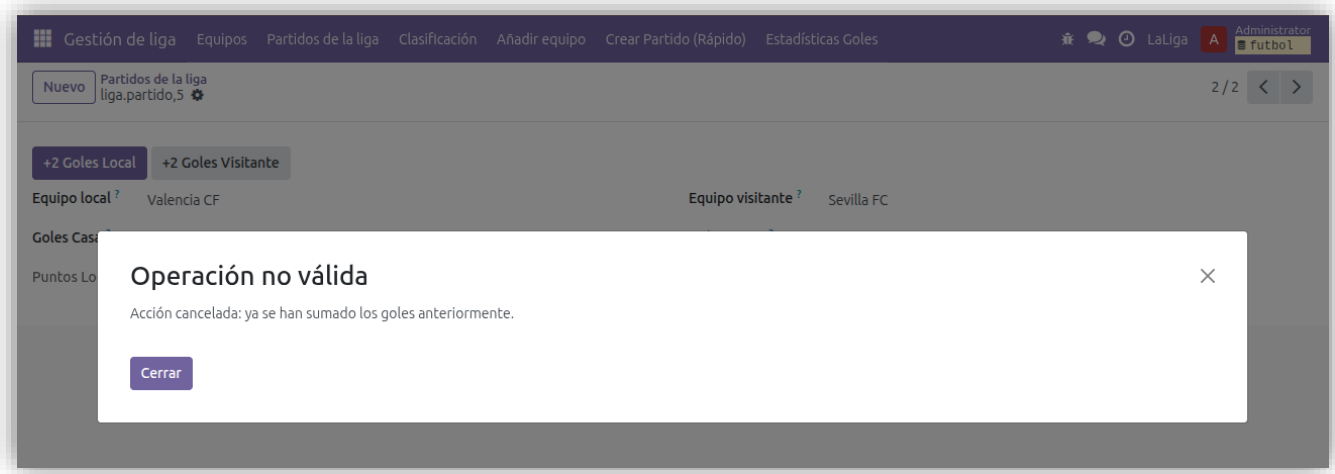
Implementación de los Botones en la Vista

views/ liga_partido_inherit_views.xml

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <record id="view_partido_form_inherit" model="ir.ui.view">
         <field name="name">liga.partido.form.inherit</field>
        <field name="model">liga.partido</field>
        <field name="inherit_id" ref="EJ07-LigaFutbol.view_partido_form"/>
        <field name="arch" type="xml">
          <xpath expr="//header" position="inside">
            <header>
                <button name="action_masive_goles_casa" string="+2 Goles Local " type="object" class="oe_highlight"/>
                <button name="action_masive_goles_visitante" string="+2 Goles Visitante (Todos)" type="object"/>
            </header>
          </xpath>

         <xpath expr="//field[@name='goles_casa']" position="after">
                <field name="puntos_casa" readonly="1"/>
                <field name="goles_casa_incrementados" invisible="1"/>
        </xpath>

        <xpath expr="//field[@name='goles_fuera']" position="after">
                <field name="puntos_fuera" readonly="1"/>
                <field name="goles_fuera_incrementados" invisible="1"/>
        </xpath>
      </field>
  </record>
<odoo>
```



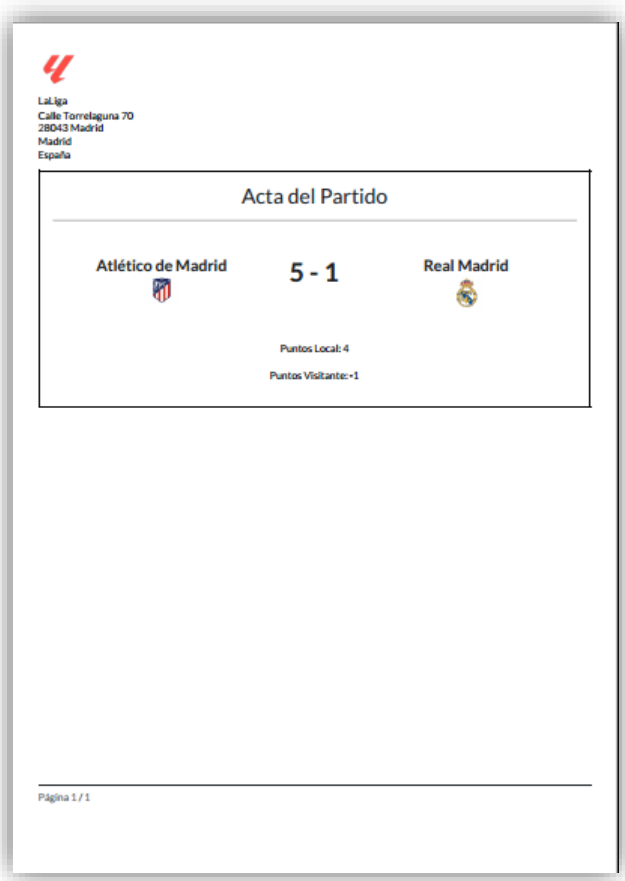
Generar informe PDF

Para generar un informe PDF en Odoo 18, se necesita definir dos elementos: la acción del informe (que crea el botón "Imprimir") y la plantilla QWeb (que define el diseño visual del documento).

Se crea el archivo en liga_futbol_pro /report/partido_report.xml y se añade en el manifest ('report/partido_report.xml')

Este código define tanto el registro del informe como el diseño que tendrá el PDF.

- Motor QWeb: el motor de plantillas QWeb para renderizar HTML que luego es convertido a PDF mediante la herramienta wkhtmltopdf.
- Binding Model: Gracias a esto, Odoo añade automáticamente un botón "Imprimir" en la vista de formulario y en la vista de lista de los partidos.
- Acceso a Datos: se utiliza la variable docs para iterar sobre los registros seleccionados, permitiendo imprimir un solo partido o varios a la vez en un único documento.





```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <record id="action_report_partido_resultado" model="ir.actions.report">
    <field name="name">Imprimir Resultado</field>
    <field name="model">liga.partido</field>
    <field name="report_type">qweb-pdf</field>
    <field name="report_name"> liga_futbol_pro.report_partido_template</field>
    <field name="report_file"> liga_futbol_pro.report_partido_template</field>
    <field name="print_report_name">'Resultado - %s' % (object.name)</field>
    <field name="binding_model_id" ref="EJ07-LigaFutbol.model_liga_partido"/>
    <field name="binding_type">report</field>
  </record>

  <template id="report_partido_template">
    <t t-call="web.html_container">
      <t t-foreach="docs" t-as="o">
        <t t-call="web.external_layout">
          <div class="page">
            <div class="text-center" style="border-bottom: 2px solid #eee; margin-bottom: 20px;">
              <h2>Acta del Partido</h2>
              <hr/>
             <div class="row" style="font-size: 24px; margin-top: 50px;">
              <div class="col-5">
                <strong t-field="o.equipo_casa.nombre"/><br/>
                <t t-if="o.equipo_casa.escudo">
                  <img t-att-src="image_data_uri(o.equipo_casa.escudo)" style="max-height: 100px;"/>
                </t>
              </div>
              <div class="col-2" style="font-size: 40px; font-weight: bold;">
                <span t-field="o.goles_casa"/> - <span t-field="o.goles_fuera"/>
              </div>
              <div class="col-5">
               <strong t-field="o.equipo_fuera.nombre"/><br/>
               <t t-if="o.equipo_fuera.escudo">
                 <img t-att-src="image_data_uri(o.equipo_fuera.escudo)" style="max-height: 100px;"/>
               </t>
             </div>
            </div>
           <div style="margin-top: 50px;">
           <p>Puntos Visitante: <span t-field="o.puntos_fuera"/></p>
           </div>
          </div>
        </t>
      </t>
    </t>
  </template>
</odoo>
```

Modelo del Wizard (asistente)

Se hace mediante un modelo de tipo TransientModel, que es una tabla temporal en la base de datos que se limpia automáticamente, ideal para ventanas emergentes.

Se crea el archivo en liga_futbol_pro/wizard/partido_wizard.py. Este archivo definirá los campos que pediremos en la ventana emergente. también un __init__.py: from . import partido_wizard

```
# -*- coding: utf-8 -*-
from odoo import models, fields, api
from odoo.exceptions import ValidationError

class PartidoWizard(models.TransientModel):
    _name = 'liga.partido.wizard'
    _description = 'Asistente para crear partidos'

    # Campos que el usuario rellenará en la ventana emergente
    equipo_casa_id = fields.Many2one('liga.equipo', string="Equipo Local", required=True)
    equipo_visitante_id = fields.Many2one('liga.equipo', string="Equipo Visitante", required=True)

    def action_crear_partido(self):
        """
        Este método se ejecuta al pulsar el botón del Wizard.
        Crea un registro real en el modelo liga.partido.
        """
        # Validación básica: no pueden jugar contra sí mismos
        if self.equipo_casa_id == self.equipo_visitante_id:
            raise models.ValidationError("¡Un equipo no puede jugar contra sí mismo!")

        # Creamos el registro en el modelo original
        self.env['liga.partido'].create({
            'equipo_casa': self.equipo_casa_id.id,
            'equipo_fuera': self.equipo_fuera_id.id,
            'goles_casa': 0,
            'goles_fuera': 0,

        })

        # Al terminar, devolvemos una acción para cerrar la ventana y refrescar la vista
        return {'type': 'ir.actions.act_window_close'}
```

Vista del Wizard (XML)

Se crea el archivo en liga_futbol_pro /wizard/partido_wizard_view.xml. Aquí se define cómo se verá la ventana (los botones "Crear" y "Cancelar").

```
<?xml version="1.0" encoding="utf-8"?>
<odoo>
  <record id="view_partido_wizard_form" model="ir.ui.view">
    <field name="name">liga.partido.wizard.form</field>
    <field name="model">liga.partido.wizard</field>
    <field name="arch" type="xml">
      <form string="Crear Nuevo Partido">
        <group>
          <field name="equipo_casa_id"/>
          <field name="equipo_fuera_id"/>
        </group>
        <footer>
          <button name="action_crear_partido" string="Crear Partido" type="object" class="btn-primary"/>
          <button string="Cancelar" class="btn-secondary" special="cancel"/>
        </footer>
      </form>
    </field>
  </record>

  <record id="action_partido_wizard" model="ir.actions.act_window">
    <field name="name">Asistente de Partidos</field>
    <field name="res_model">liga.partido.wizard</field>
    <field name="view_mode">form</field>
    <field name="target">new</field> </record>

 <menuitem id="menu_liga_partido_wizard"
          name="Crear Partido (Rápido)"
          parent="liga_futbol.liga_base_menu"
          action="action_liga_partido_wizard"
          sequence="20"/>
</odoo>
```

Configurar los archivos __init__.py:

- En actividad_01/__init__.py: from . import models, wizard
- En actividad_01/wizard/__init__.py: from . import partido_wizard

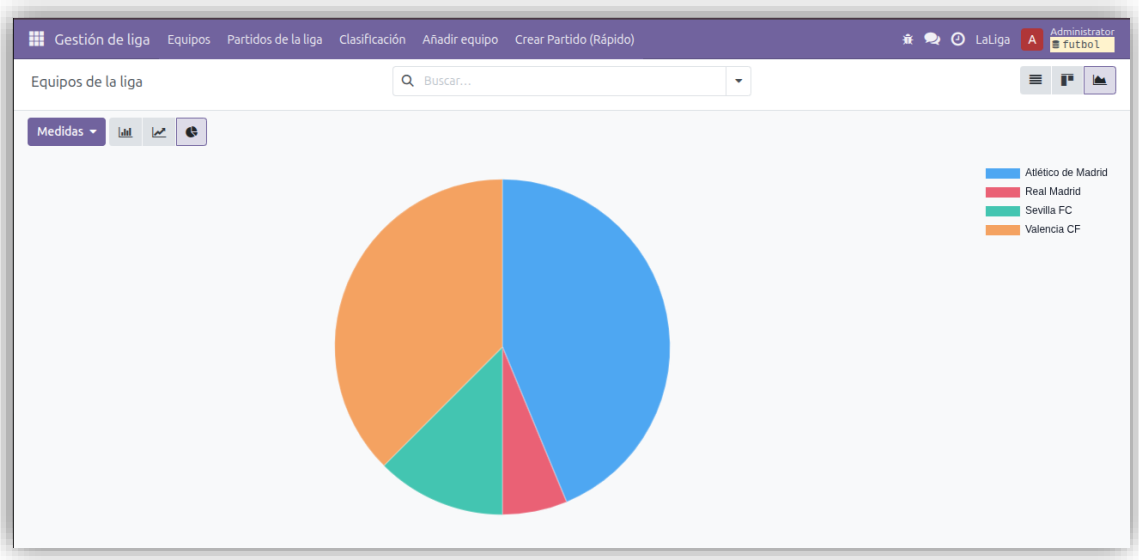
Y en el __manifest__.py, añade en data : 'wizard/ liga_partido_inherit_view.xml',

Vista Graph

Añadir al archivo liga_futbol_pro /views/ liga_partido_inherit_views.xml:

```
<record id="view_partido_graph" model="ir.ui.view">
      <field name="name">liga.partido.graph</field>
      <field name="model">liga.partido</field>
      <field name="arch" type="xml">
        <graph string="Análisis de Goles" type="bar" sample="1">
          <field name="equipo_casa" type="row"/>
          <field name="goles_casa" type="measure"/>
        </graph>
      </field>
    </record>

    <record id="action_liga_partido_grafico_pro" model="ir.actions.act_window">
      <field name="name">Estadísticas Goles Casa</field>
      <field name="res_model">liga.partido</field>
      <field name="view_mode">graph,list,form</field>
      <field name="view_id" ref="view_partido_graph"/>
      <field name="context">{'graph_measure': 'goles_casa'}</field>
    </record>
```



Configuración de Seguridad (security/ir.model.access.csv)

Permisos de lectura/escritura sobre el modelo TransientModel:

```
id,name,model_id:id,group_id:id,perm_read,perm_write,perm_create,perm_unlink
access_liga_partido_wizard,liga.partido.wizard,model_liga_partido_wizard,base.group_user,1,1,1,1
```

Web Controller

Los controladores gestionan las rutas URL y la comunicación desde el exterior hacia Odoo.

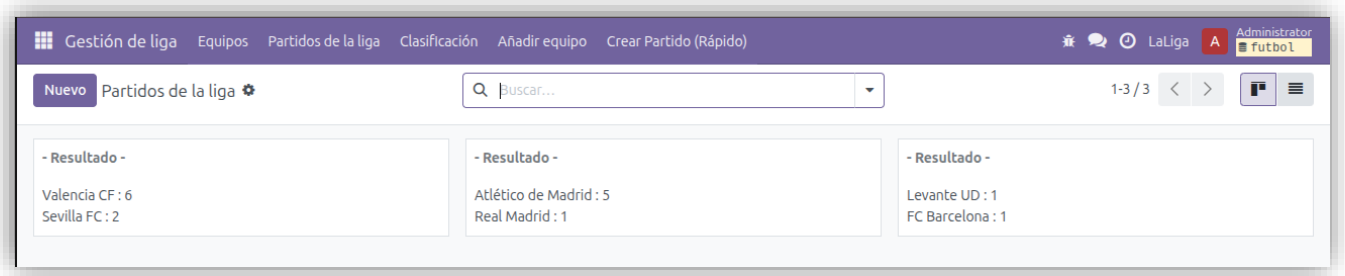
En Controllers/ __init__.py: from . import main.py

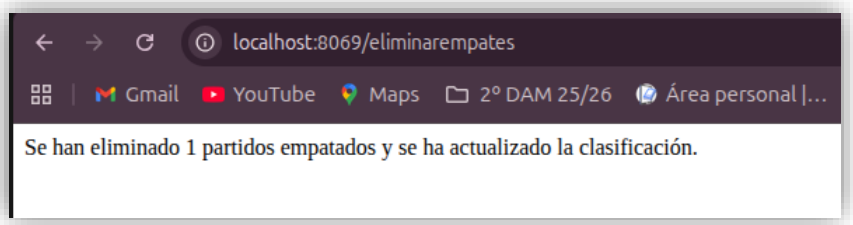
Controllers/ main.py

```
# -*- coding: utf-8 -*-
from odoo import http
from odoo.http import request

class LigaController(http.Controller):
    # Definimos la URL local: http://localhost:8069/eliminarempates
    @http.route('/eliminarempates', type='http', auth='user', website=False)
    def eliminar_empates(self):
        # Buscamos todos los partidos
        partidos = request.env['liga.partido'].sudo().search([])
        eliminados = 0

        for p in partidos:
            if p.goles_casa == p.goles_fuera:
                p.unlink() # Borramos el partido
                eliminados += 1
        # Recalcular la clasificación tras borrar
        # Llamamos al método que ya existe en el modelo original
        request.env['liga.partido'].sudo().actualizoRegistrosEquipo()
        return "Se han eliminado %s partidos empatados y se ha actualizado la clasificación." % eliminados
```





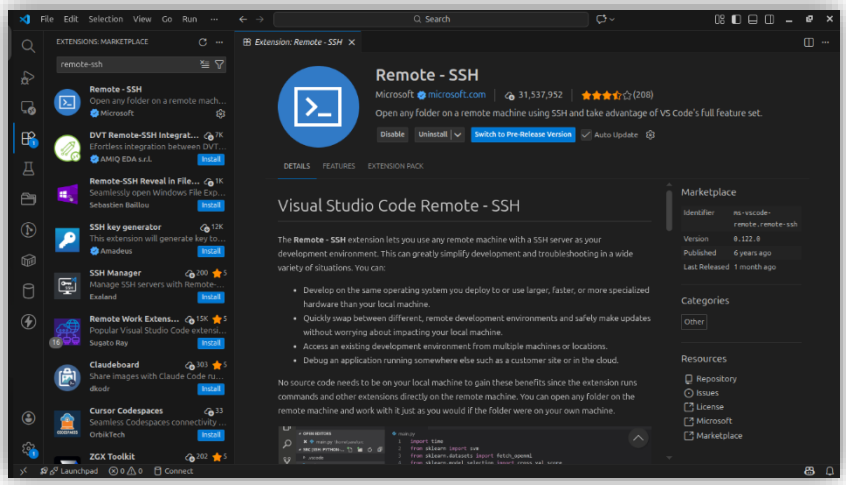
ACTIVIDAD 02

Haz un vídeo de no más de 3 minutos de duración donde explicas y demuestras cómo has probado todas las opciones (crear, modificar, consultar, eliminar de la API Resto del ejemplo "EJ08-API-REST-Socios" con una herramienta similar a "PostMan" o la extensión de Visual Studio Code "Thunder Client".

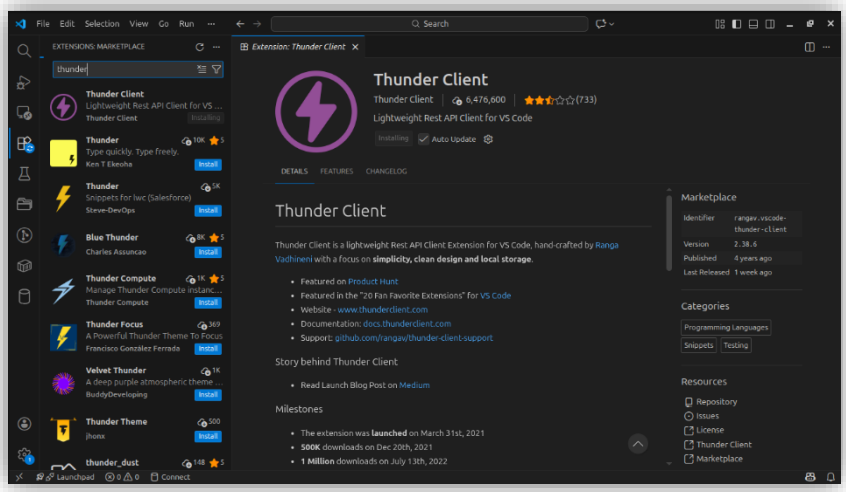
Nota: el vídeo puede realizarse en cualquier idioma (Valenciano, Castellano o inglés) y no requiere de una gran edición, sólo se valorará que se demuestran las pruebas y se detalle que se está haciendo y por qué.

Preparación del Entorno

- **Configurar el acceso SSH en VS Code**
  - Instala la extensión Remote - SSH de Microsoft para poder trabajar en VM.
  - Presionar F1 o Ctrl+Shift+P y selecciona Remote-SSH: Connect to Host....
  - Escribir usuario@ip-de-tu-vm → miriam@127.0.0.1
  - Una vez conectado, abrir la carpeta del proyecto en la VM (EJ08-API-REST-Socios).



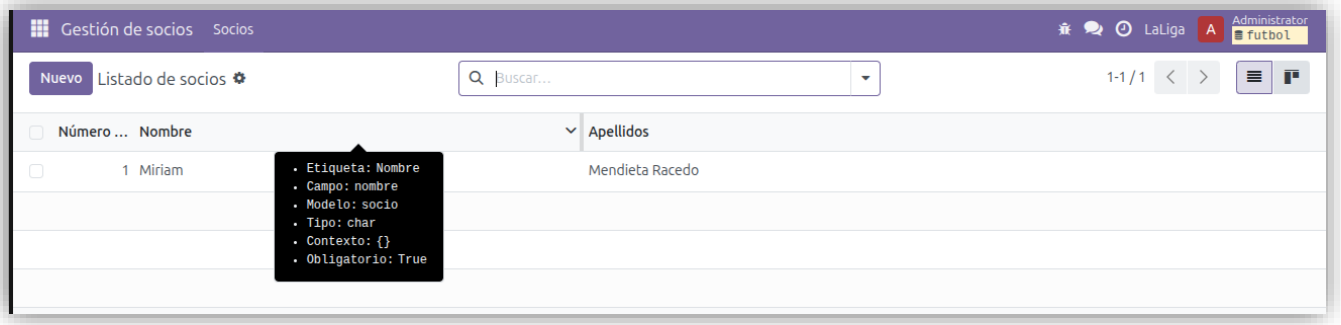
- **Instalar y Configurar Thunder Client (Modo Remoto)**
  - Cuando se está conectado por SSH, las extensiones que instales se ejecutan en la VM.
  - Buscar Thunder Client en el panel de extensiones e instalar.
  - Hacer clic en el icono del rayo en la barra lateral para abrir Thunder Client y crear New Request.



Inspección de Metadatos mediante el ORM

Cómo encontrar los nombres técnicos de los campos y la url para las peticiones:

Para saber qué campos puedes enviar (como name, email, etc.), en el modo desarrollador de Odoo, buscar formulario a trabajar y sobre la etiqueta de un campo; aparecerá un cuadro negro indicando el Nombre técnico a utilizar.



En Odoo, cada campo se ve en pantalla está definido en una clase de Python que hereda de models.Model.

Otra manera de saber que campos se requiere es ir a Ajustes → Técnico → Estructura de base de datos → Modelos → Buscar el modelo (ej. socios).

Allí se obtiene la lista completa de campos, si son obligatorios (required) y sus valores por defecto.

En cuanto a la construcción técnica de la URL (Endpoint), Odoo no es una API REST pura por defecto, sino que utiliza una estructura de Controladores de Ruta. En la carpeta controllers se busca .py (ej: apirest.py) el decorador que nos indica los elementos para poner en la url:

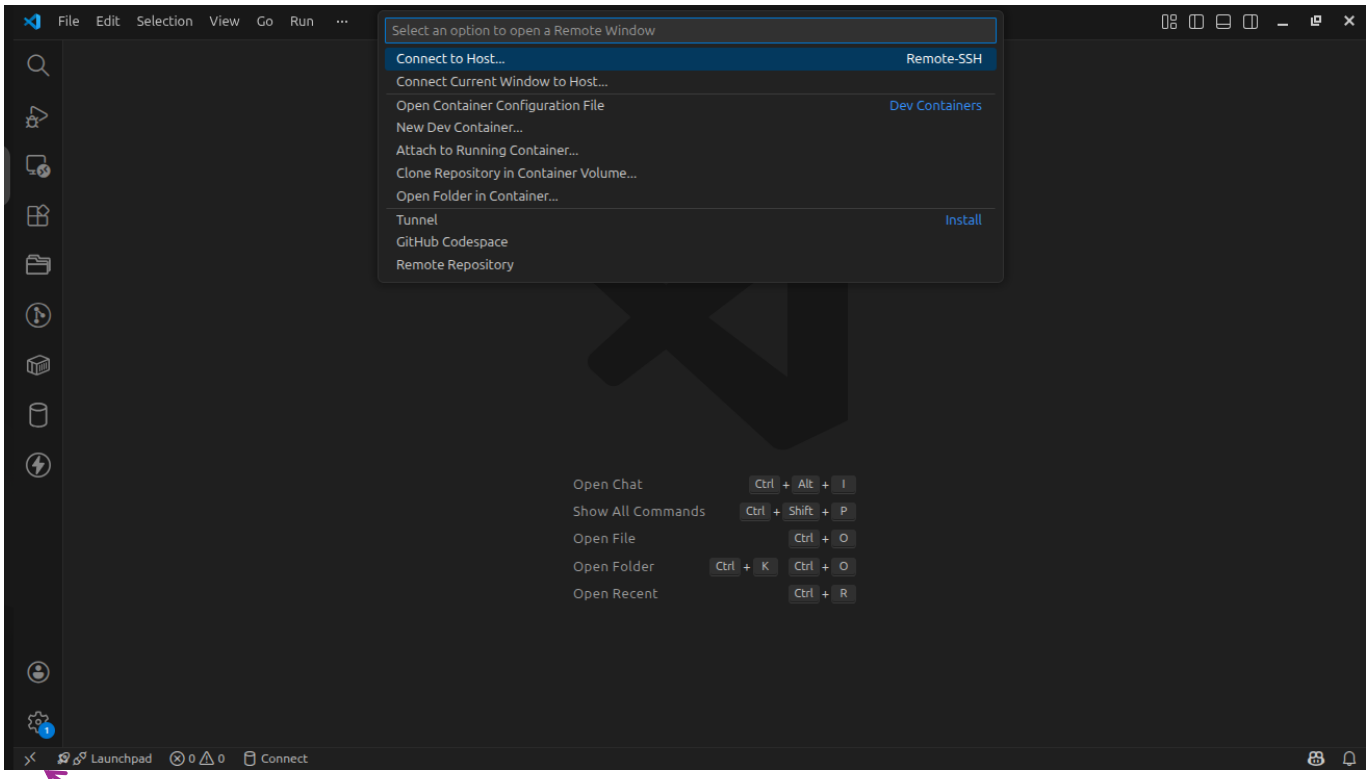
@http.route('/gestion/apirest/<model>', auth="none", cors='*', csrf=False,

methods=["POST", "PUT", "PATCH"], type='http')

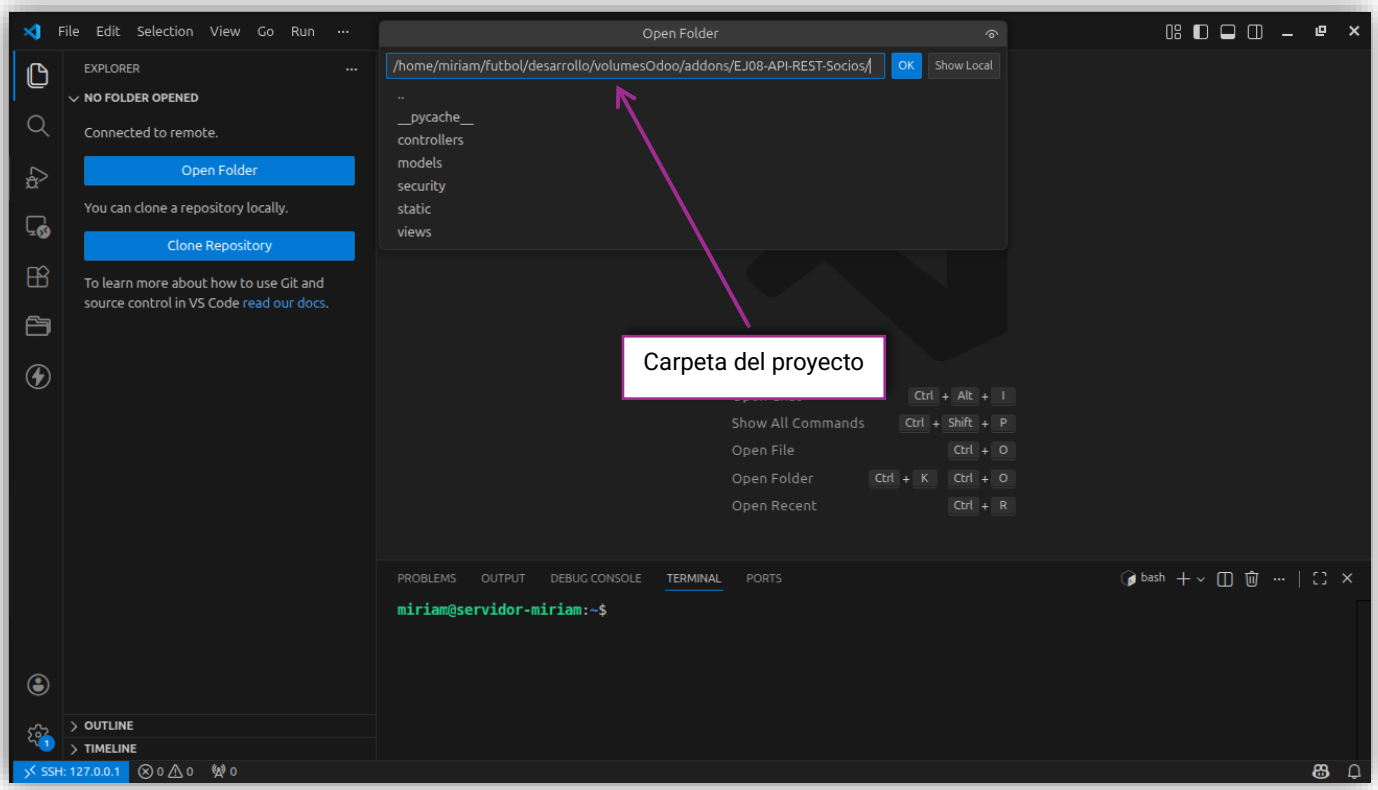
En este caso es una ruta dinámica (/gestion/apirest/<model>): Odoo utiliza un sistema de Werkzeug Routing. El segmento <model> actúa como una variable de ruta que se inyecta directamente como argumento en la función de Python. Esto permite que un solo método maneje cualquier modelo (ej: res.partner, sale.order, socio).

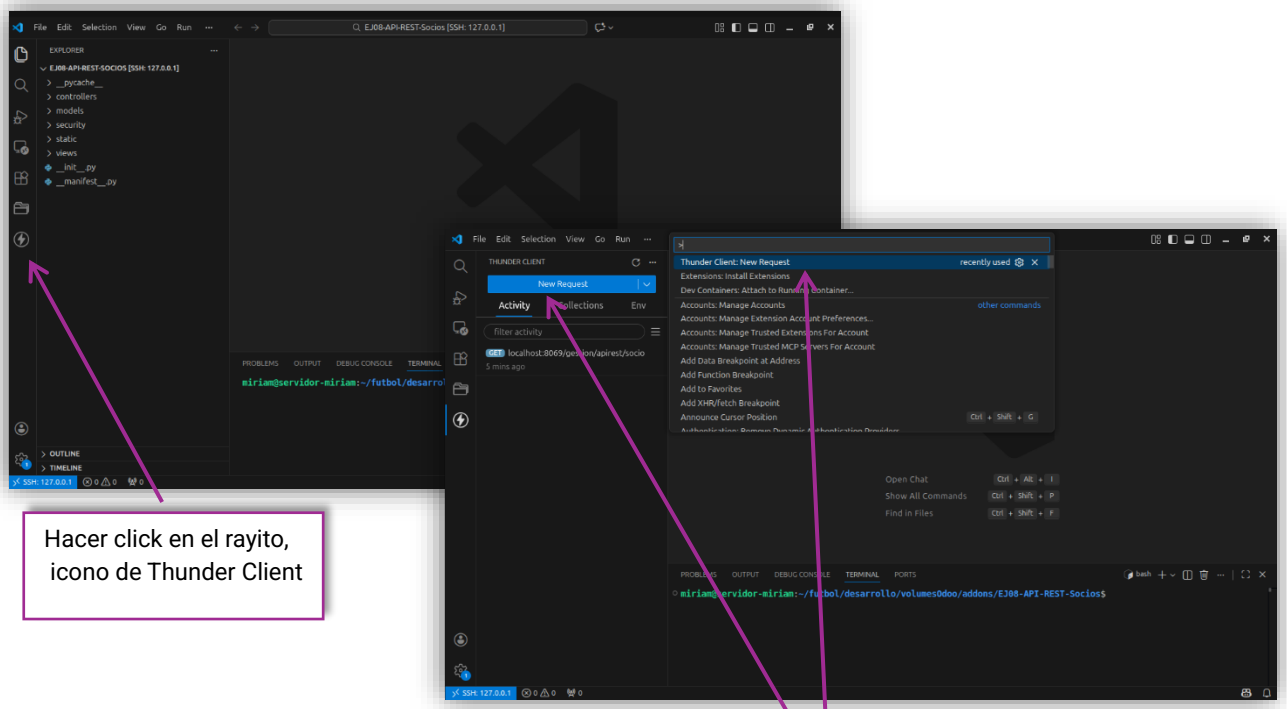
La URL final se construye uniendo la IP de la VM (o localhost por el túnel SSH) con la ruta del decorador:

Endpoint → http://localhost:8069/gestion/apirest/socio



Cilcar en este icono para conexión SSH a la VM





Hacer click en el rayito, icono de Thunder Client

Clicar En “New Request” o Ctrl+shipt+p: Thunder New Request

Peticiones en Thunder Client

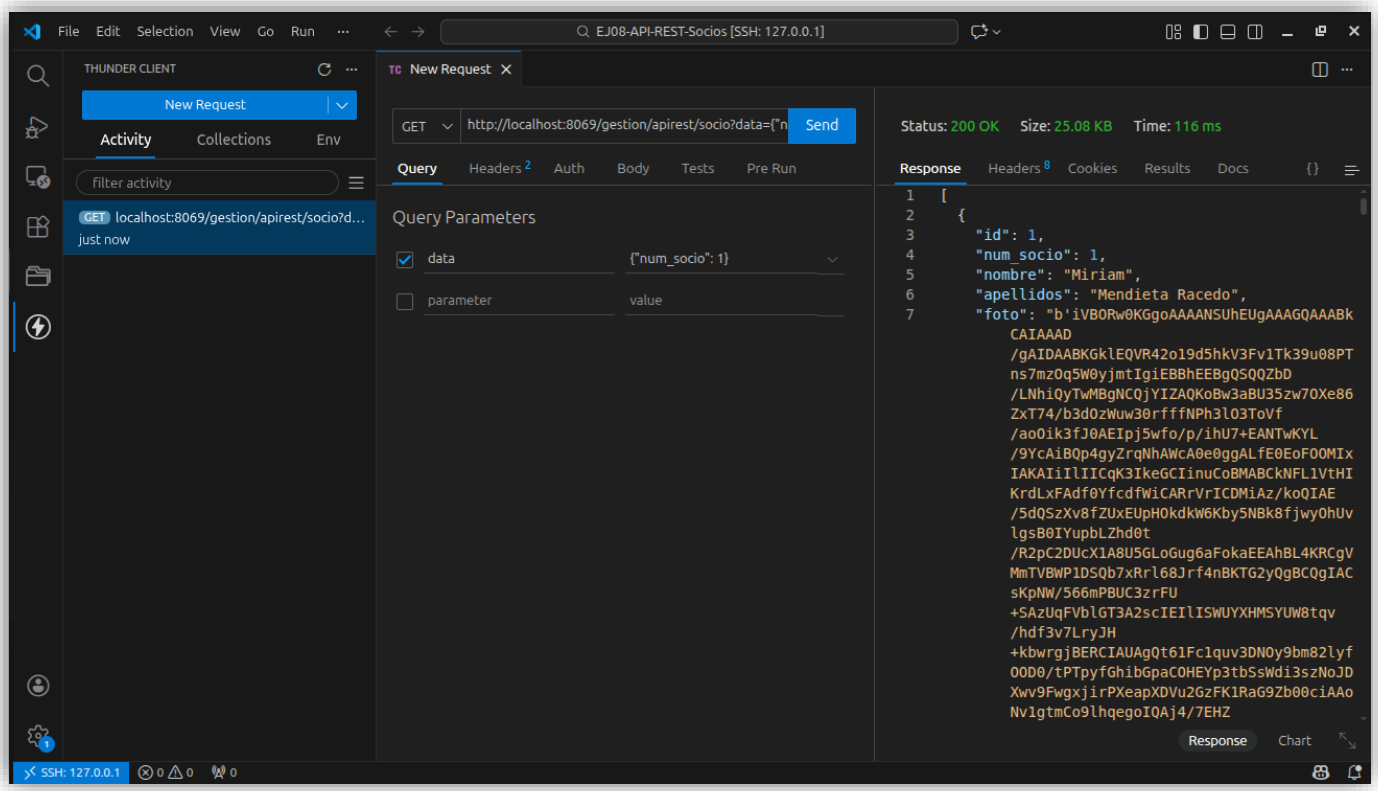
A) Consultar (GET) - Leer de cada socio

Esta operación recupera registro por socio, requiere que se pase en la url data num_socio. Se puede hacer directamente en url:

http://localhost:8069/gestion/apirest/socio?data={"num_socio":1}

también se puede poner en la url ***http://localhost:8069/gestion/apirest/socio*** y en el panel Query poner en parameter → ***data*** value →***{"num_socio":1}***

- **Método:** GET
- **URL:** http://localhost:8069/gestion/socio
- Presiona **Send**. Se recibe un JSON con la lista de socios y un código 200 OK.



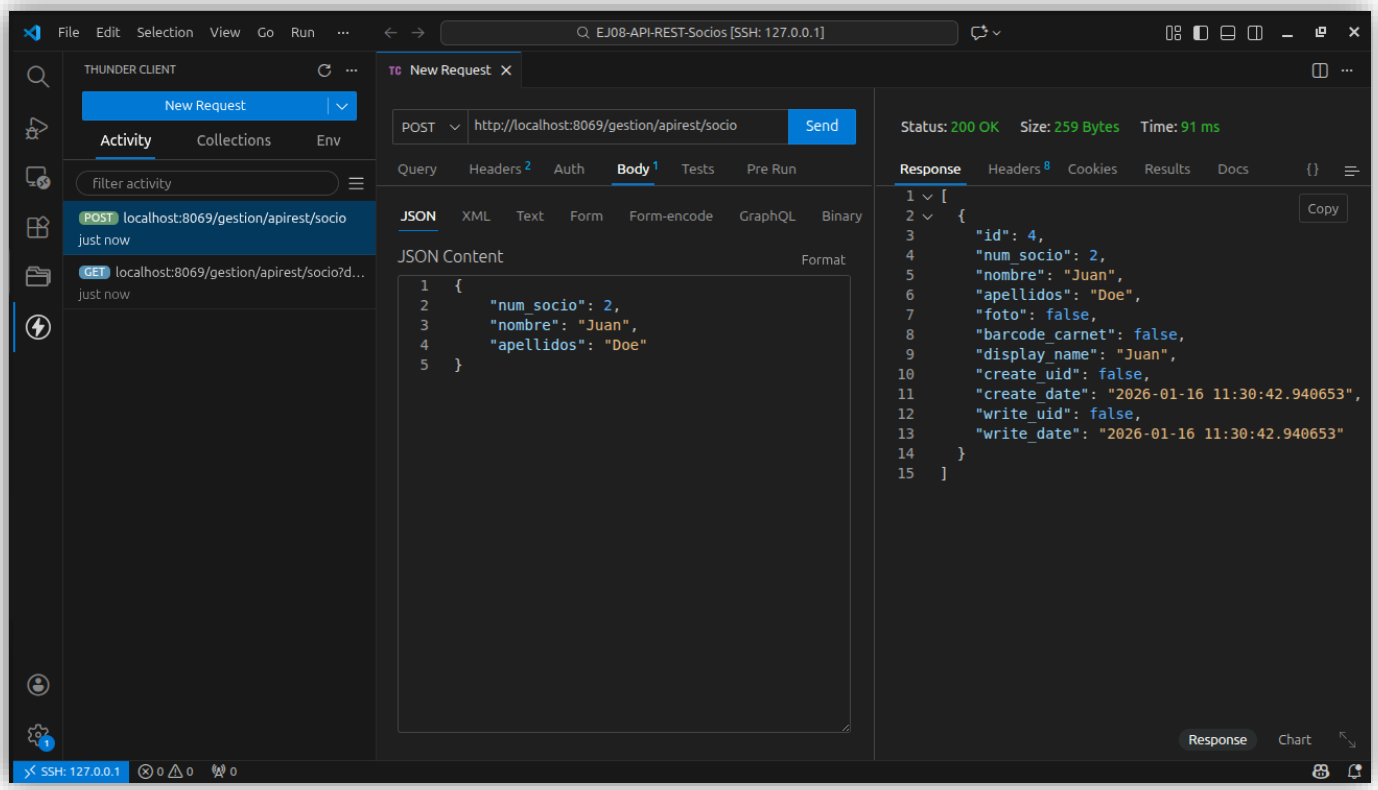
B) Crear (POST) - Añadir un nuevo socio

Para crear, se debe enviar los datos en el cuerpo (Body) de la petición.

- **Método:** POST
- **URL:** http://localhost:8069/gestion/apirest/socio
- **Body (JSON):**

```
{
    "num_socio": "2",
    "nombre": "Juan",
    "apellidos": "Doe",
}
```

- **Resultado esperado:** 200 OK.

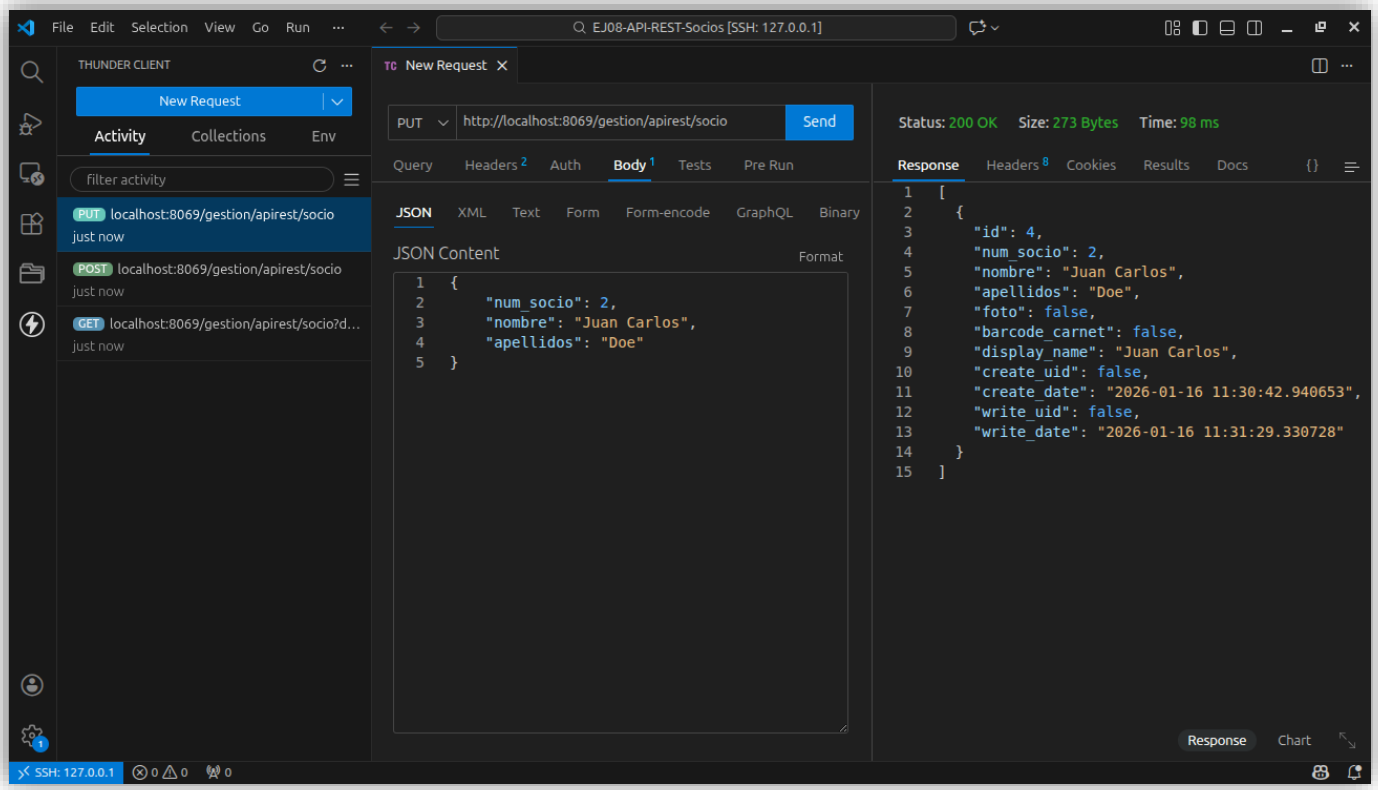


C) Modificar (PUT/PATCH) - Actualizar un socio

- **Método:** PUT (o PATCH)
- **URL:** http://localhost:8069/gestion/apirest/socio
- **Body (JSON):**

```
{
    "num_socio": "2",
    "nombre": "Juan Carlos",
    "apellidos": "Doe",
}
```

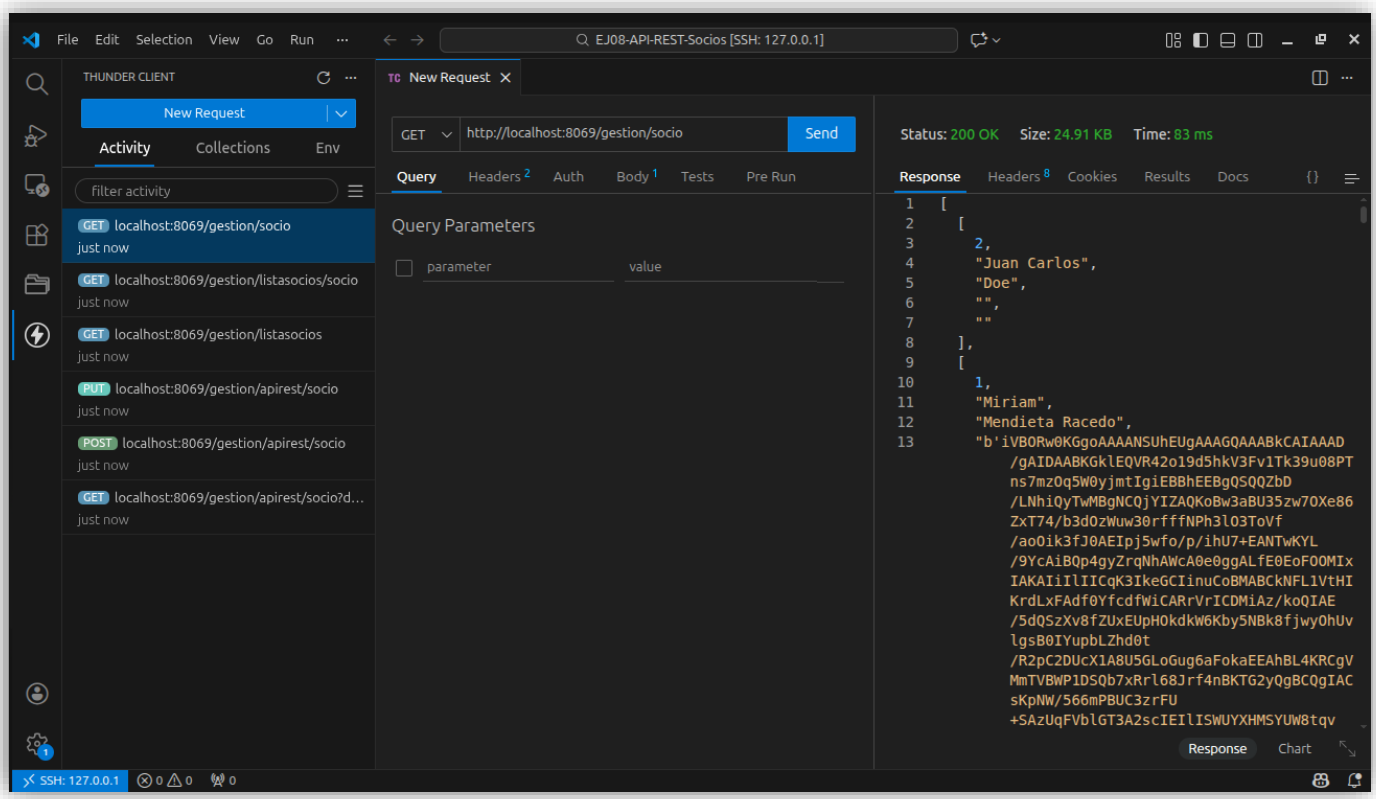
- **Resultado esperado:** 200 OK con los datos actualizados.



D) Consultar (GET) - Leer todos los socios

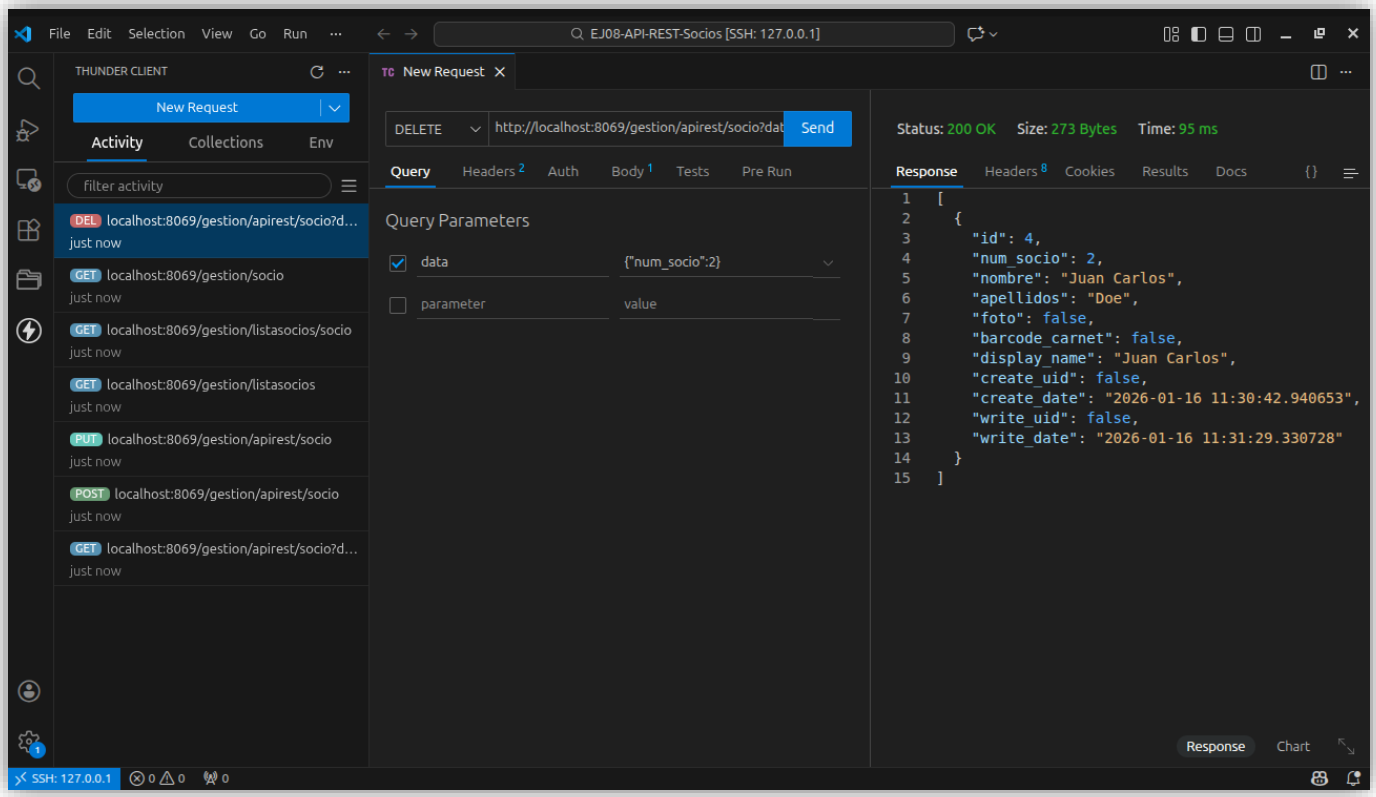
Esta operación recupera la lista de registros.

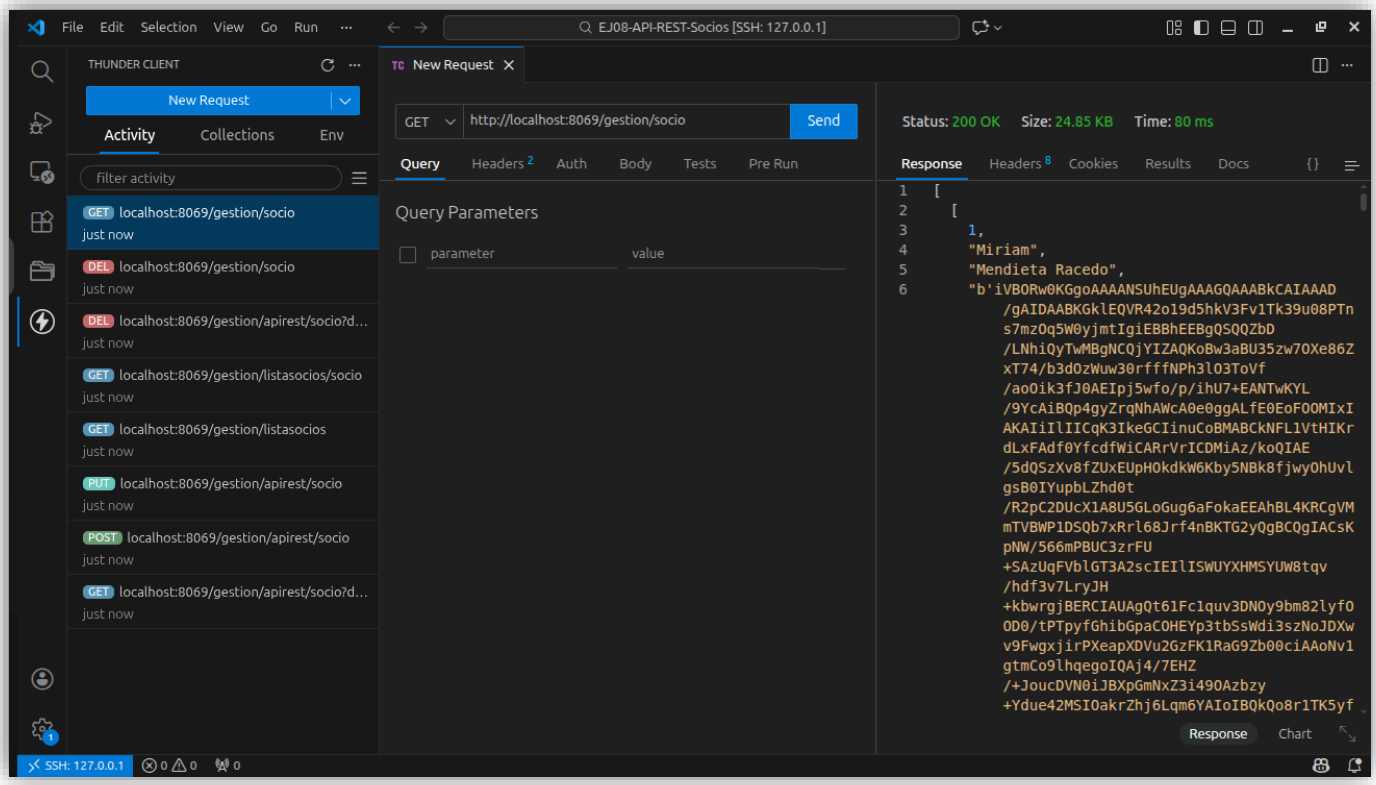
- Método: GET
- URL: http://localhost:8069/gestion/socio
- Presiona Send. Se recibe un JSON con la lista de socios y un código 200 OK.



E) Eliminar (DELETE) - Borrar un socio

- **Método:** DELETE
- **URL:** ***http://localhost:8069/gestion/apirest/socio?data={"num_socio":1}***
- **Resultado esperado:** 204 No Content o 200 OK confirmando el borrado.


File Edit Selection View Go Run
EJ08-API-REST-Socios [SSH: 127.0.0.1]
THUNDER CLIENT
New Request
Activity Collections Env
filter activity
DEL localhost:8069/gestion/apirest/socio?d...
just now
GET localhost:8069/gestion/socio
just now
GET localhost:8069/gestion/listasocios/socio
just now
GET localhost:8069/gestion/listasocios
just now
PUT localhost:8069/gestion/apirest/socio
just now
POST localhost:8069/gestion/apirest/socio
just now
GET localhost:8069/gestion/apirest/socio?d...
just now
New Request
DELETE
http://localhost:8069/gestion/apirest/socio?dat
Send
Query Headers 2 Auth Body 1 Tests Pre Run
Query Parameters
data
{"num_socio":2}
parameter
value
Status: 200 OK Size: 273 Bytes Time: 95 ms
Response Headers 8 Cookies Results Docs
{
"id": 4,
"num_socio": 2,
"nombre": "Juan Carlos",
"apellidos": "Doe",
"foto": false,
"barcode_carnet": false,
"display_name": "Juan Carlos",
"create_uid": false,
"create_date": "2026-01-16 11:30:42.940653",
"write_uid": false,
"write_date": "2026-01-16 11:31:29.330728"
}
]
Response Chart
SSH: 127.0.0.1


File Edit Selection View Go Run
EJ08-API-REST-Socios [SSH: 127.0.0.1]
THUNDER CLIENT
New Request
Activity Collections Env
filter activity
GET localhost:8069/gestion/socio
just now
DEL localhost:8069/gestion/socio
just now
DEL localhost:8069/gestion/apirest/socio?d...
just now
GET localhost:8069/gestion/listasocios/socio
just now
GET localhost:8069/gestion/listasocios
just now
PUT localhost:8069/gestion/apirest/socio
just now
POST localhost:8069/gestion/apirest/socio
just now
GET localhost:8069/gestion/apirest/socio?d...
just now
New Request
GET
http://localhost:8069/gestion/socio
Send
Query Headers 2 Auth Body Tests Pre Run
Query Parameters
parameter
value
Status: 200 OK Size: 24.85 KB Time: 80 ms
Response Headers 8 Cookies Results Docs
[
[
1,
"Miriam",
"Mendieta Racedo",
"b'iVBORw0KGgoAAAANSUhEUgAAAGQAAABkCAIAAAD
/gAIDAABKGklEQVR42o19d5hkV3Fv1Tk39u08PTn
s7mzOq5W0yjmtIgiEBBhEEBgQSQQZbD
/LNhiQyTwMBgNCQjYIZAQKoBw3aBU35zw7OXe86Z
xT74/b3dOzWuw30rfffNPh3lO3ToVf
/aoOik3fJ0AEIpj5wfo/p/ihU7+EANTwKYL
/9YcAiBQp4gyZrqNhAWcA0e0ggALfE0EoFOOMIxI
AKAIiIlIICqK3IkeGCIinuCoBMABCkNFL1VtHIKr
dLxFAdf0YfcdfWiCARrVrICDMiAz/koQIAE
/5dQSzXv8fZUxEUpHOkdkW6Kby5NBk8fjwyOhUvl
gsB0IYupbLZhd0t
/R2pC2DUcX1A8U5GLoGug6aFokaEEAhBL4KRCgVM
mTVBWP1DSQb7xRrl68Jrf4nBKTG2yQgBCQgIACsK
pNW/566mPBUC3zrFU
+SAzUqFVblGT3A2scIEIlISWUYXHMSYUW8tqv
/hdf3v7LryJH
+kbwrgjBERCIAUAgQt61Fc1quv3DNOy9bm82lyfO
OD0/tPTpyfGhibGpaCOHEYp3tbSsWdi3szNoJDXw
v9FwgxjirPXeapXDVu2GzFK1RaG9Zb00ciAAoNv1
gtmCo9lhqegoIQAj4/7EHZ
/+JoucDVN0iJBXpGmNxZ3i49OAzbzy
+Ydue42MSIOakrZhj6Lqm6YAIoIBQkQo8r1TK5yf
Response Chart
SSH: 127.0.0.1

Listado de los códigos que pueden aparecer en cada método (GET, POST, PUT, DELETE)

2xx: *Éxito*

- **200 OK:** La petición ha tenido éxito (GET y PUT).
- **201 Created:** Se suele devolver tras un **POST** exitoso cuando se crea un nuevo socio.
- **204 No Content:** Se usa a veces en el **DELETE** si el servidor borra el registro pero no tiene nada que devolverte.

3xx: *Redirecciones*

- **301 Moved Permanently:** La URL ha cambiado.
- **303 See Other:** Muy común en Odoo cuando después de una acción te redirige a la página de "Log in" o a la vista de lista.

4xx: *Errores del Cliente* (Algo falta en la petición)

Son los más importantes para corregir en **Thunder Client**.

- **400 Bad Request:** * "SOCIONOINDICADO": el JSON está mal escrito o falta un campo obligatorio.
- **401 Unauthorized:** No se ha iniciado sesión o no se tiene permisos.
- **403 Forbidden:** indica un problema de privilegios de acceso. (permisos de acceso/ACLs).
- **404 Not Found:** La URL no existe en el archivo .py.
- **405 Method Not Allowed:** se intenta hacer un DELETE en una ruta que solo acepta GET.

5xx: *Errores del Servidor* (Fallo en el código Python)

- **500 Internal Server Error:** Un error de indentación, una coma que falta o intentar acceder a un campo que no existe en el modelo de Odoo.
- **503 Service Unavailable:** Odoo está apagado o reiniciándose.

ACTIVIDAD 03

Pon en marcha en tu máquina el ejemplo “EJ08-API-REST-Socios”. Después crea en una aplicación Python, que ejecutarás en tu máquina que permitirá lanzar órdenes a un bot de Telegram para crear, modificar, borrar y consultar registros de Socios.

El bot de Telegram por cada petición consultará el API Rest de “EJ08-API-REST-Socios”.
Las órdenes que deberá soportar el bote de Telegram son:

- Crear, número=”número”, apellidos=”apellidos”, num_socio=”numerosocio”
- Modificar, número=”número”, apellidos=”apellidos”, num_socio=”numerosocio”
- Consultar, num_socio=socio=”

En caso de recibir una orden distinta, devolverá el mensaje “Orden no soportada”.

Para crear el bot Telegram debes utilizar BotFather:

https://planetachatbot.com/como-crear-bot-para-telegram-y-darle-funcionalidad/

Para realizar la aplicación, te serán útiles las siguientes bibliotecas:
La biblioteca “request”, para consumir una API Rest en Python:

- https://j2logo.com/python/python-requests-peticiones-http/
- https://www.nylas.com/blog/use-python-requests-module-rest-apis/

La biblioteca “Python Telegram Bot” para crear un Bot de Telegram

https://github.com/python-telegram-bot/python-telegram-bot/

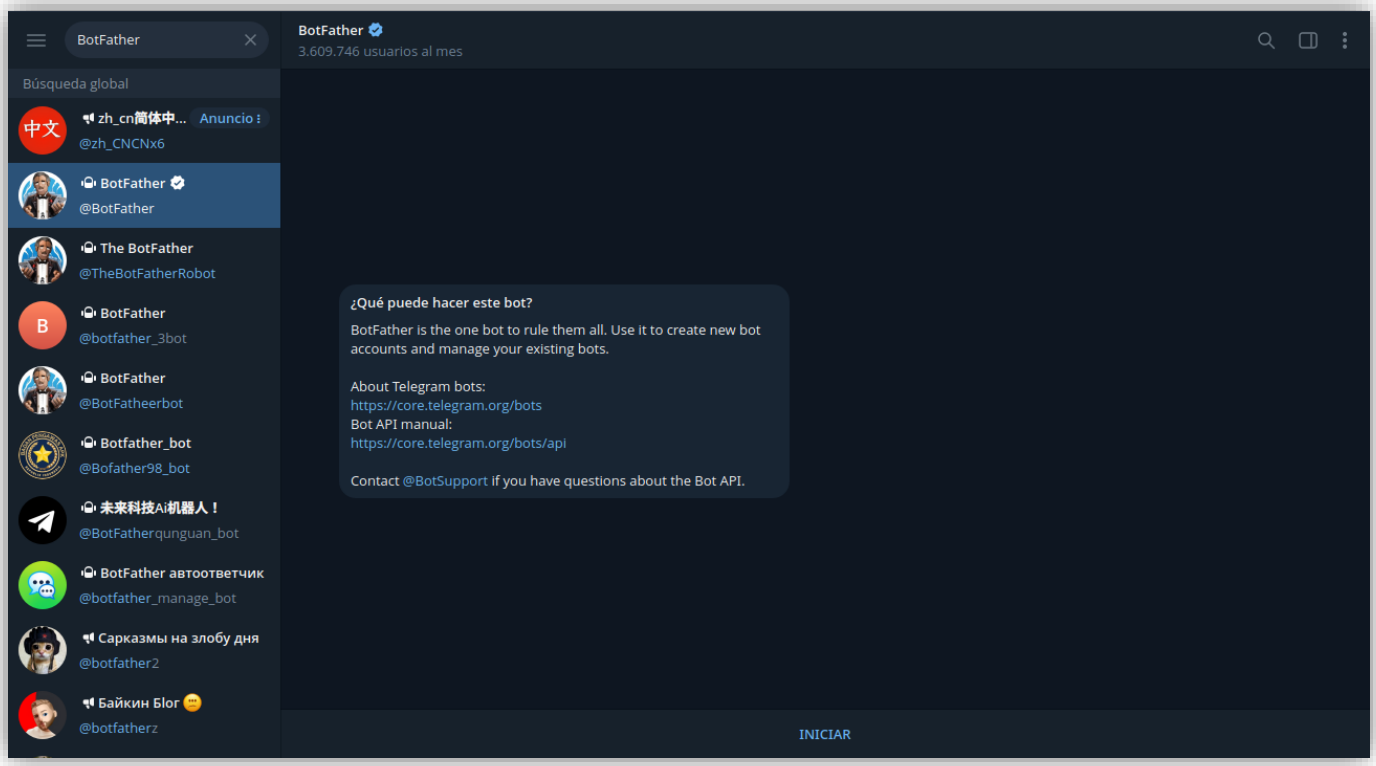
https://github.com/python-telegram-bot/python-telegram-bot/tree/master/examples

Configuración del Entorno

1. Instalar Dependencias

   sudo apt update
   sudo apt install python3 python3-pip
   pip3 install python-telegram-bot requests

2. Crear un Bot en Telegram
   Abrir Telegram y buscar "BotFather".



Hacer clic en el perfil de BotFather y luego clic en el botón "Iniciar" o escribe /start en el chat.

Crear un Nuevo Bot utilizando el comando: ***/newbot***

Dar un nombre al bot.

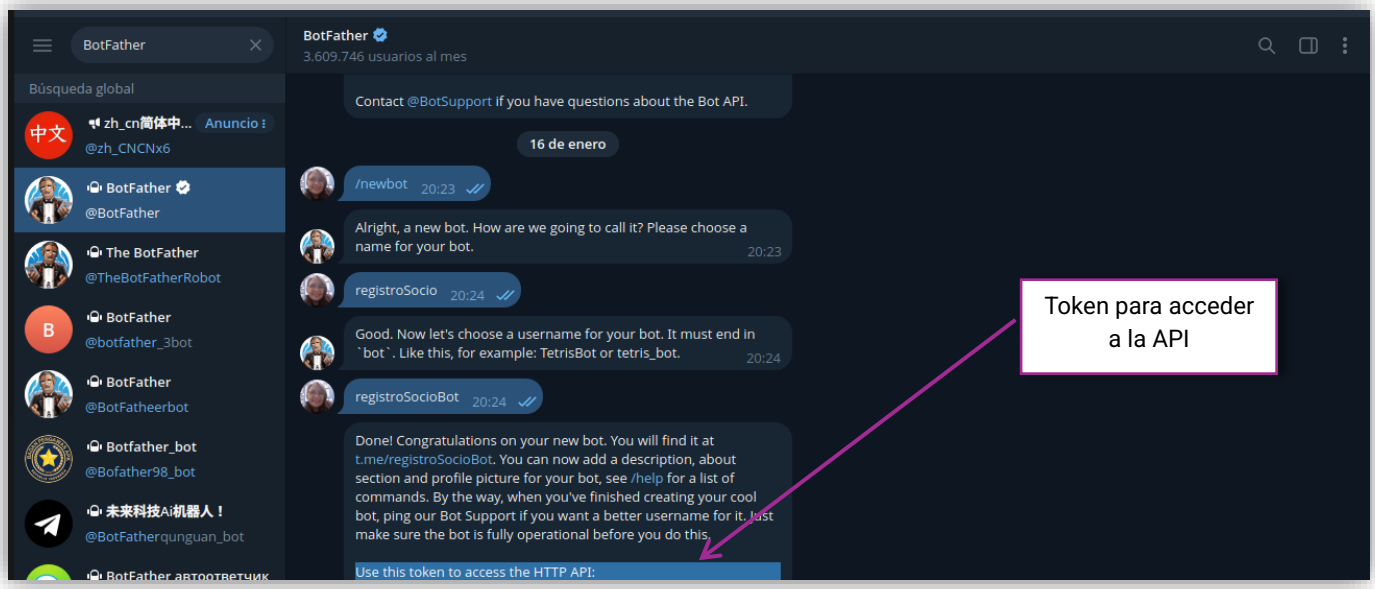
Después, elegir un nombre de usuario único para el bot (registroSocio). Debe terminar con "bot" (por ejemplo,RegistroSocioBot). Si el nombre de usuario ya está en uso, BotFather te lo indicará, y deberás elegir otro.

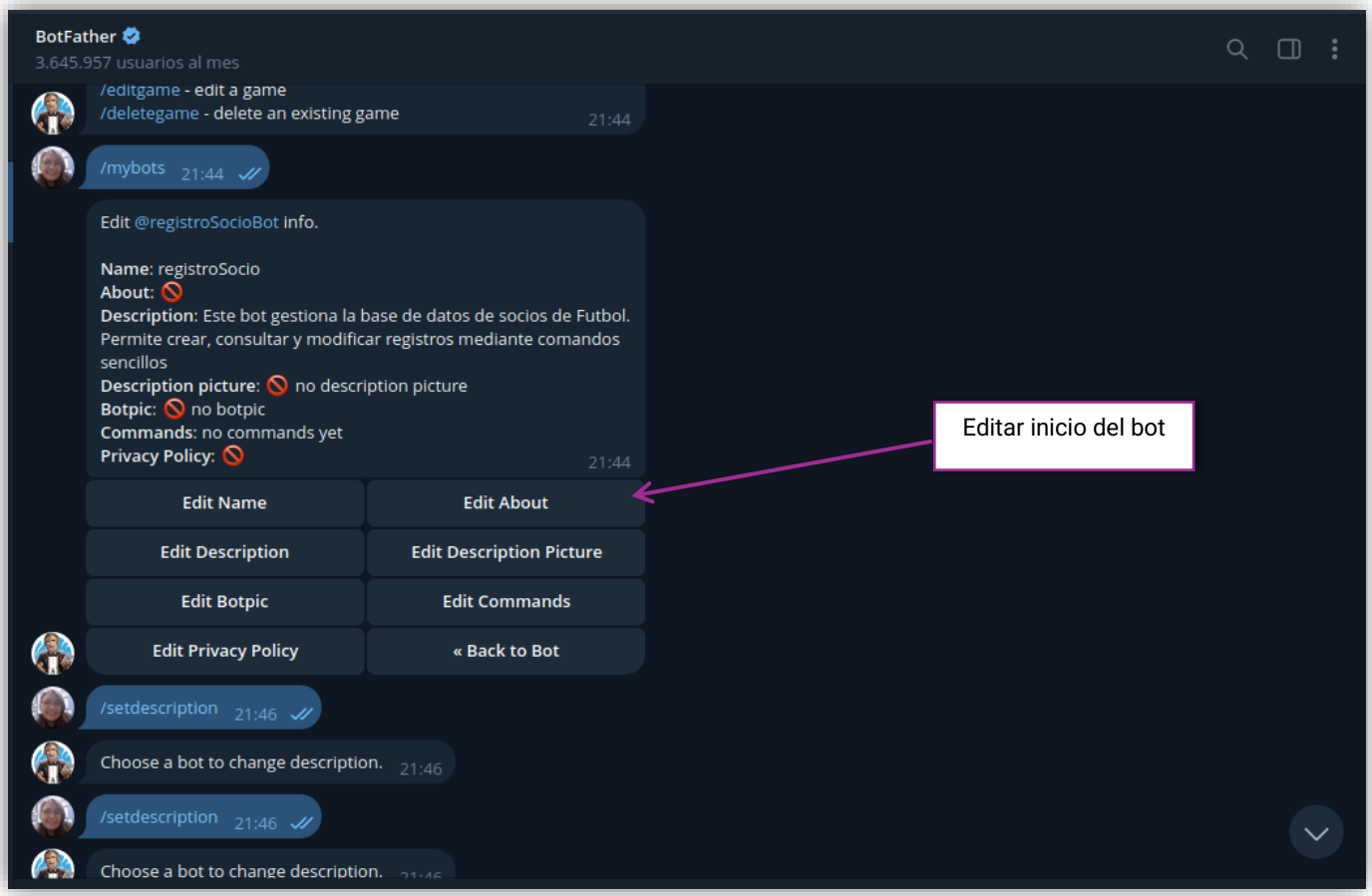
Una vez creado el bot, BotFather te proporcionará un token de API que se verá algo así:

Aquí tienes tu nuevo bot: https://t.me/TuNombreDeUsuarioBot

Usa este token para acceder a la API: 123456789:ABCdefGhIJKlmnoPQRstuVWXyz

Copia el token, ya que se necesita para conectarte al bot desde la aplicación de Python.



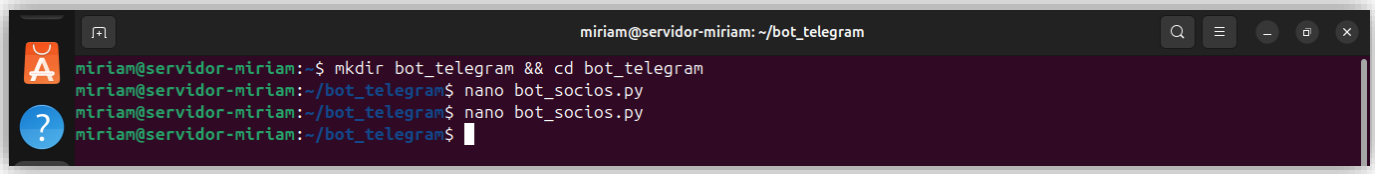


Crear la Aplicación en Python

mkdir bot_telegram

cd bot_telegram

nano bot_socios.py



```
import requests
import logging
import json
from telegram import Update
from telegram.ext import ApplicationBuilder, ContextTypes, CommandHandler, MessageHandler, filters

# --- CONFIGURACIÓN ---
TOKEN = 'TOKEN_TELEGRAM'
API_URL = 'http://localhost:8069/gestion/apirest/socio'

logging.basicConfig(level=logging.INFO)

# --- BIENVENIDA ---
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    mensaje = (
        "📊 **Sistema de Gestión de Socios**\n\n"
        "Para interactuar con el sistema, por favor utilice los siguientes formatos:\n\n"
        " 🔹 **Registro de nuevo socio:**\n"
        "`Crear, num_socio=..., nombre=..., apellidos=...` \n\n"
        " 🔹 **Consulta de registro:**\n"
        "`Consultar, num_socio=...` \n\n"
        " 🔹 **Actualización de datos:**\n"
        "`Modificar, num_socio=..., nombre=..., apellidos=...` \n\n"
        "**Nota:** Es imprescindible respetar la estructura de campos separados por comas."
    )

    await update.message.reply_text(mensaje, parse_mode='Markdown')
 --- PROCESAR MENSAJES ---
async def procesar_orden(update: Update, context: ContextTypes.DEFAULT_TYPE):
    texto = update.message.text

    try:
        # Limpieza de entrada
        partes = [p.strip() for p in texto.split(',')]
        comando = partes[0].lower()

        # parámetros
        params = {}
        for p in partes[1:]:
            if '=' in p:
                clave, valor = p.split('=', 1)
                params[clave.strip()] = valor.strip()


        num = params.get('num_socio')
        nom = params.get('nombre')
        ape = params.get('apellidos')

        res = ""
```

```
    # --- VALIDACIONES ---
# CONSULTAR (Solo requiere num_socio)
    if comando == "consultar":
        if not num:
            res = " **Error:** Debe indicar `num_socio=...` para consultar."
        else:
            payload_json = json.dumps({"num_socio": num})
            r = requests.get(API_URL, params={'data': payload_json})

            if r.status_code == 200:
                datos = r.json()
                if isinstance(datos, list) and len(datos) > 0:
                    s = datos[0]
                    res = (f" **Socio Encontrado:**\n\n"
                           f" 🔹 **Nº:** {s.get('num_socio', 'N/A')}\n"
                           f" 🔹 **Nombre:** {s.get('nombre', 'N/A')}\n"
                           f" 🔹 **Apellidos:** {s.get('apellidos', 'N/A')}")
                else:
                    res = f" No existe ningún socio con el número `{num}`."

    #CREAR y MODIFICAR (todos los campos)
    elif comando in ["crear", "modificar"]:
        if not all([num, nom, ape]):
            res = " **Campos incompletos:** Para esta acción debe indicar `num_socio`, `nombre` y `apellidos`."
        else:
            if comando == "crear":
                r = requests.post(API_URL, json=params)
            else:
                r = requests.put(API_URL, json=params)

            data_api = r.json()
            estado = data_api.get('estado') if isinstance(data_api, dict) else ""

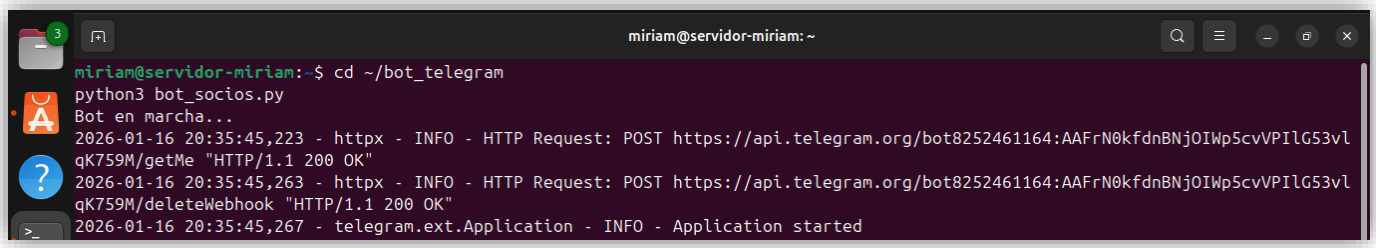
            if estado == 'REPETIDO':
                res = f" **Error:** El número de socio `{num}` ya está en uso."
            elif estado == 'SOCIONOINDICADO':
                res = " **Error:** El sistema no reconoce el número de socio enviado."
            elif r.status_code in [200, 201]:
                accion = "registrado" if comando == "crear" else "actualizado"
                res = f" **Éxito:** Socio {accion} correctamente."
            else:
                res = f"**Error: número de socio repetido o formato incorrecto."

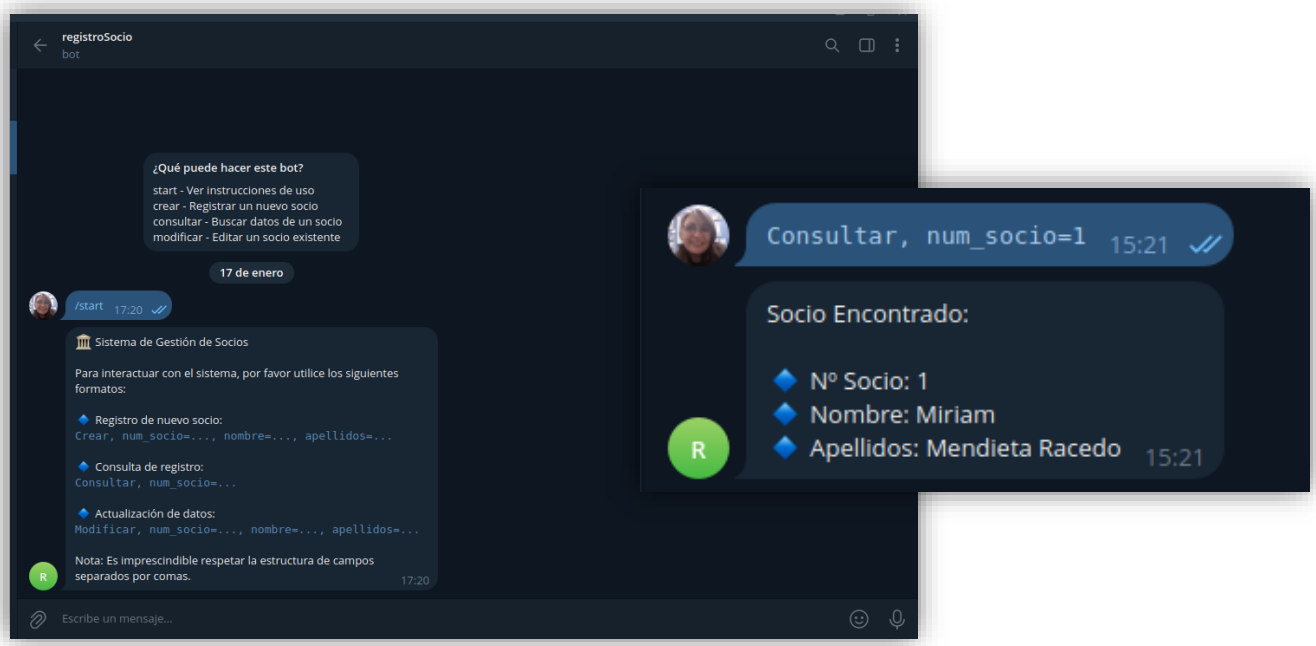
    else:
        res = "**Comando no válido.** Use: Crear, Consultar o Modificar."

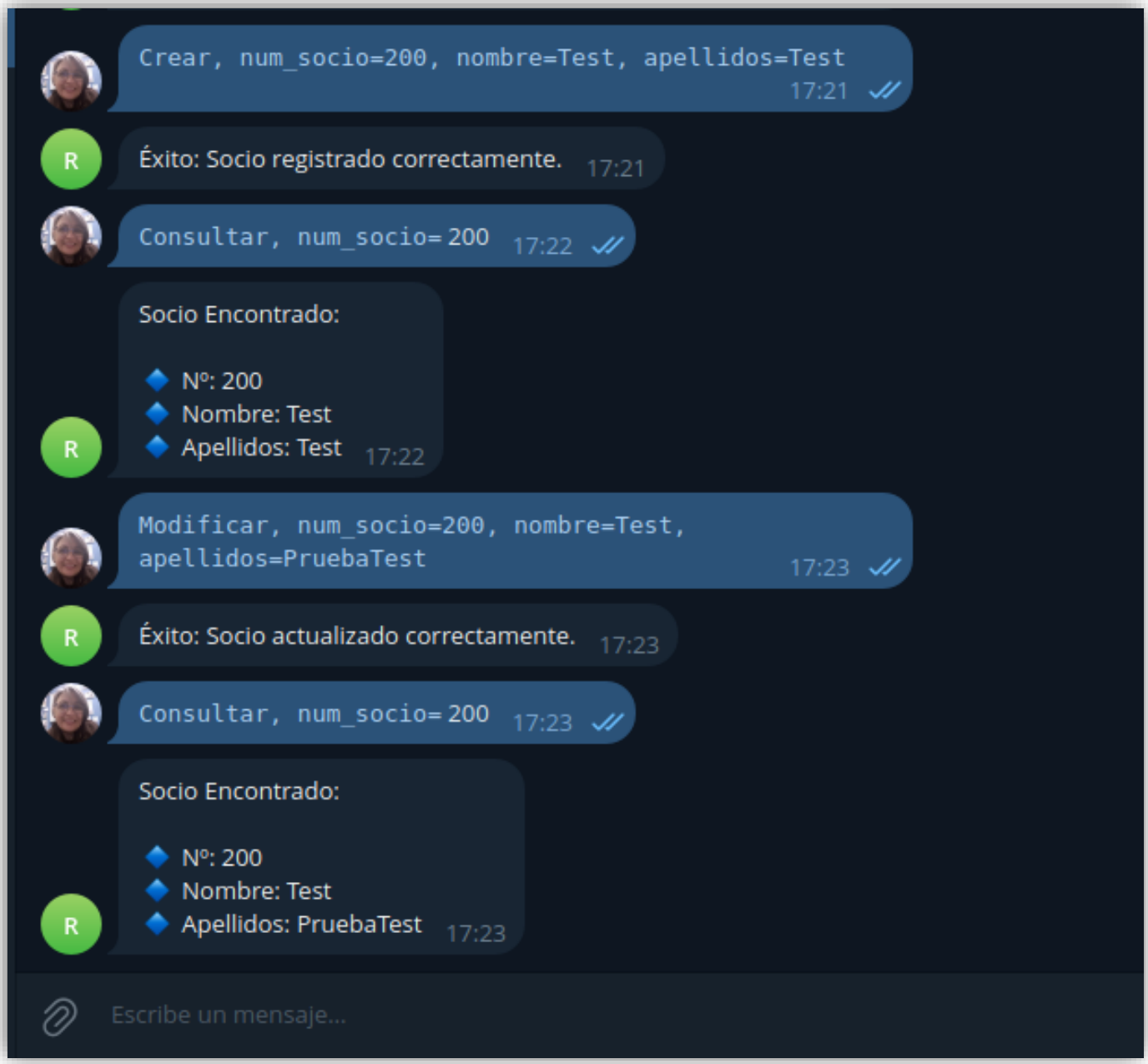
        # ENVIAR RESPUESTA
    await update.message.reply_text(res, parse_mode='Markdown')
```

```
    except Exception as e:
        logging.error(f"Error: {e}")
        await update.message.reply_text("**Error de sintaxis:** Asegúrese de separar los campos con comas y usar el formato `cl>

# --- INICIO ---
if __name__ == '__main__':
    app = ApplicationBuilder().token(TOKEN).build()
    app.add_handler(CommandHandler("start", start))
    app.add_handler(MessageHandler(filters.TEXT & (~filters.COMMAND), procesar_orden))
    print("Servidor de Gestión de Socios iniciado...")
    app.run_polling()
```






Crear, num_socio=200, nombre=Test, apellidos=Test
17:21
Éxito: Socio registrado correctamente. 17:21
Consultar, num_socio= 200 17:22
Socio Encontrado:
Nº: 200
Nombre: Test
Apellidos: Test 17:22
Modificar, num_socio=200, nombre=Test, apellidos=PruebaTest
17:23
Éxito: Socio actualizado correctamente. 17:23
Consultar, num_socio= 200 17:23
Socio Encontrado:
Nº: 200
Nombre: Test
Apellidos: PruebaTest 17:23
Escribe un mensaje...

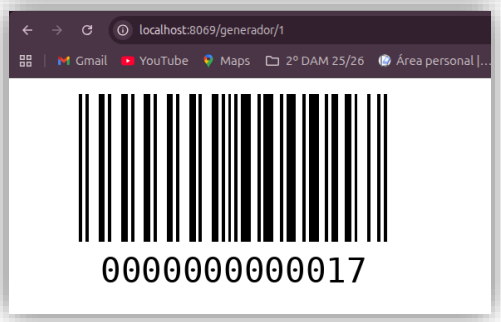

Consultar, num_socio=123457895 17:39
No existe ningún socio con el número 123457895. 17:39
Consultar, num_socio= 17:40
Error: Debe indicar num_socio=... para consultar. 17:40
Crear, num_socio=, nombre=., apellidos= 17:41
Campos incompletos: Para esta acción debe indicar num_socio, nombre y apellidos. 17:41

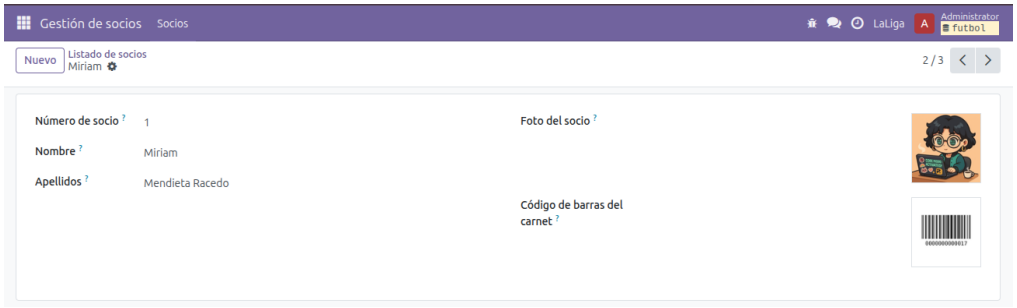
ACTIVIDAD 04

De forma similar al ejemplo “EJ09-GenerarBarcode”, haz un Web Contoller que se apoya en una herramienta externa PIL: https://pypi.org/project/Pillow/ que genere en cada petición una imagen formada por píxeles aleatorios, recibiendo como parámetros el ancho y largo de la imagen deseada.

Puedes ayudarte del código del ejemplo “EJ09-GenerarBarcode” y con ejemplos de código como https://www.daniweb.com/programming/software-development/threads/488949/looking-for-random-image-generator-pil-script y también para guardar la imagen como texto para mostrarla en base64 en https://stackoverflow.com/questions/646286/how-to-write-png-image-to-string-with-the-pil

```
miriam@servidor-miriam:~/futbol/desarrollo$ docker exec -u root -it desarrollo-web-1 pip3 install python-barcode python-barcode[
images] Pillow --break-system-packages
```





Preparación del entorno

1. Instalar librería Pillow → ***pip install Pillow***
   Al estar en VM con Docker se utiliza el siguiente código:

```
miriam@servidor-miriam: ~/futbol/desarrollo
miriam@servidor-miriam:~/futbol/desarrollo$ docker exec -u root -it desarrollo-web-1 pip3 install Pillow --break-system-packages
WARNING: Skipping /usr/lib/python3.12/dist-packages/charset_normalizer-3.3.2.dist-info due to invalid metadata entry 'name'
Requirement already satisfied: Pillow in /usr/lib/python3/dist-packages (10.2.0)
WARNING: Skipping /usr/lib/python3.12/dist-packages/charset_normalizer-3.3.2.dist-info due to invalid metadata entry 'name'
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manag
er. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv
miriam@servidor-miriam:~/futbol/desarrollo$
```

2. Crear la estructura del módulo:

```
generadorPillow/
    ├─── __init__.py            # Contiene: from . import controllers
    ├─── __manifest__.py        # (Metadatos y dependencias como Pillow)
    └─── controllers/
    ├─── __init__.py            # Contiene: from . import generarpillow
    └─── generarpillow.py       # Código del Web Controller con PIL
```

El módulo utiliza el micro-framework de controladores de Odoo, basado en Werkzeug, para exponer una ruta dinámica. La generación gráfica se delega a la librería Pillow, específicamente utilizando el módulo ImageDraw para crear primitivas geométricas como rectángulos, círculos y líneas.

Una vez que se genera la imagen, el flujo de bytes se codifica utilizando el estándar Base64, lo que permite insertar la imagen directamente en el DOM de una página web como una cadena de texto. Esto facilita mostrar la imagen sin necesidad de almacenarla en el servidor, optimizando el proceso de carga y visualización.

- generadorPillow/__init__.py:

```
# -*- coding: utf-8 -*-
from . import controllers
```

- generadorPillow/__manifest__.py

```
{
    'name': "Generador de Imágenes Aleatorias",
    'summary': "Genera una imagen de píxeles aleatorios",
    'description': "Módulo que expone una ruta para generar imágenes aleatorias usando Pillow.",
    'author': "Miriam Mendieta.-",
    'website': "http://tusitio.com",
    'category': 'Tools',
    'version': '1.0',
    'depends': ['base'],
    'data': [],
    'installable': True,
    'application': True,
    'license': 'LGPL-3',
}
```

- generadorPillow/controllers/__init__.py

```
# -*- coding: utf-8 -*-
from . import generarpillow
```

- generadorPillow/controllers/generarpillow.py

```
import io
import base64
import random
from PIL import Image, ImageDraw
from odoo import http
from odoo.http import request

class GeneradorMultipatronAleatorio(http.Controller):

    @http.route('/generar-imagen/<int:ancho>/<int:largo>', auth='public', type='http')
    def crear_patron_aleatorio(self, ancho, largo, **kw):
        # 1. Validación de seguridad
        if ancho > 2000 or largo > 2000:
            return "Dimensiones demasiado grandes (máximo 2000px)."

        # 2. Crear lienzo
        image = Image.new('RGB', (ancho, largo))
        draw = ImageDraw.Draw(image)

        # 3. Elegir un tipo de patrón aleatorio
        patrones_disponibles = ['rayas', 'degradado', 'pixel_art', 'circulos']
        patron_elegido = random.choice(patrones_disponibles)

        titulo_patron = "" # Para el título del HTML

        # Lógica para dibujar el patrón
        if patron_elegido == 'rayas':
            titulo_patron = "Rayas Aleatorias"
            grosor = random.randint(15, 60) # Grosor aleatorio para las rayas
            for y in range(0, largo, grosor):
                color_rgb = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255))
                draw.rectangle([0, y, ancho, y + grosor], fill=color_rgb)
```

```
    elif patron_elegido == 'degradado':
        titulo_patron = "Degradado Suave"
        color1 = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255))
        color2 = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255))
        for i in range(largo):
            r = int(color1[0] + (color2[0] - color1[0]) * i / largo)
            g = int(color1[1] + (color2[1] - color1[1]) * i / largo)
            b = int(color1[2] + (color2[2] - color1[2]) * i / largo)
            draw.line([(0, i), (ancho, i)], fill=(r, g, b))

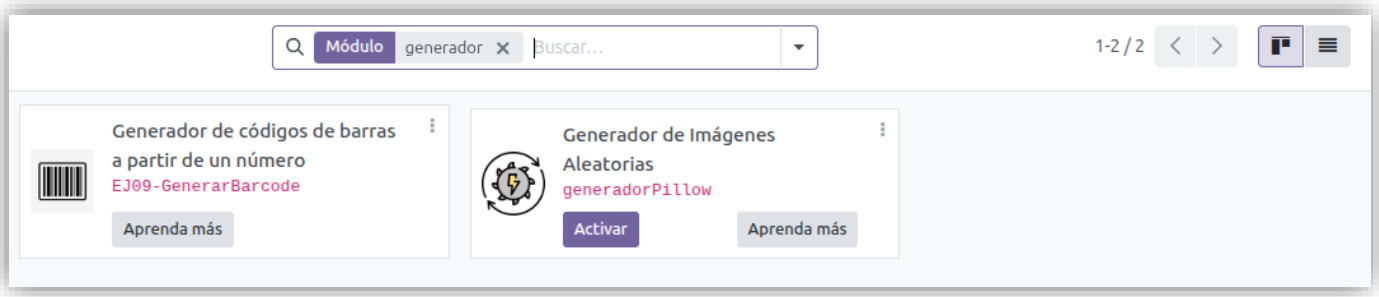
    elif patron_elegido == 'pixel_art':
        titulo_patron = "Pixel Art Aleatorio"
        # Creamos una imagen mucho más pequeña y la escalamos
        factor_bloque = random.randint(10, 40) # Tamaño aleatorio para los "píxeles"
        img_pequena = Image.new('RGB', (ancho // factor_bloque, largo // factor_bloque))
        pixeles = img_pequena.load()
        for x_p in range(img_pequena.width):
            for y_p in range(img_pequena.height):
                pixeles[x_p, y_p] = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255))
        image = img_pequena.resize((ancho, largo), resample=Image.NEAREST) # Se reemplaza la imagen principal

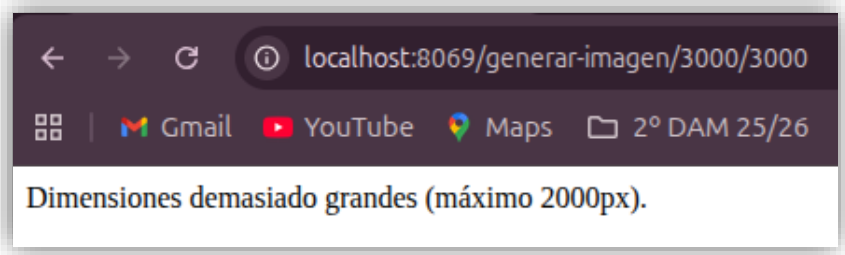
    elif patron_elegido == 'circulos':
        titulo_patron = "Círculos Aleatorios"
        # Podemos usar RGBA para transparencias si queremos que se superpongan
        image = Image.new('RGBA', (ancho, largo), (255, 255, 255, 255)) # Fondo blanco
        draw = ImageDraw.Draw(image)
        num_circulos = random.randint(5, 20)
        for _ in range(num_circulos):
            x = random.randint(0, ancho)
            y = random.randint(0, largo)
            radio = random.randint(min(ancho, largo) // 10, min(ancho, largo) // 3)
            color_rgb = (random.randint(0, 255), random.randint(0, 255), random.randint(0, 255), random.randint(80, 180))
            draw.ellipse([x - radio, y - radio, x + radio, y + radio], fill=color_rgb)

    # 4. Guardar en buffer de memoria
    buffered = io.BytesIO()
    image.save(buffered, format="PNG")

    # 5. Convertir a Base64
    img_str = base64.b64encode(buffered.getvalue()).decode("utf-8")
```

```
# 6. Retornar HTML completo
return f'''
   <div style="text-align:center; padding:20px; font-family: sans-serif; background-color: #f8f8f8;">
      <h3 style="color:#333;">Generador de Patrones: {titulo_patron} ({ancho}x{largo})</h3>
      <img src="{img_str}"
          style="border:1px solid #ddd; box-shadow: 0 4px 10px rgba(0,0,0,0.15); max-width:100%; height:auto;" />
      <br><br>
      <button onclick="window.location.reload();"
            style="padding: 10px 20px; font-size: 16px; cursor: pointer;
                  background-color: #007bff; color: white; border: none;
                  border-radius: 5px; box-shadow: 0 2px 5px rgba(0,0,0,0.1);">
         Generar Nuevo Patrón
      </button>
   </div>
'''
```






localhost:8069/generar-imagen/300/300
YouTube
Maps
2º DAM 25/26
Área personal |...
Inicio | Microso...
RECURSOS O...
Correo: MENDI...
Generador de Patrones: Círculos Aleatorios (300x300)
Generar Nuevo Patrón


Generador de Patrones: Rayas Aleatorias (300x300)
Generar Nuevo Patrón


Generador de Patrones: Pixel Art Aleatorio (300x300)
Generar Nuevo Patrón


Generador de Patrones: Degradado Suave (300x300)
Generar Nuevo Patrón


Generador de Patrones: Rayas Aleatorias (300x300)
Generar Nuevo Patrón

BIBLIOGRAFIA

- Apuntes del Profesor.
- Python para todos. Explorando la información con Python 3. Charles R. Severance
- https://docs.python.org/es/3/library/copy.html
- https://www.youtube.com/watch?v=ewTvHFLnU1M&list=PL5ESsgnHGfa8d3Eet-muUA8quawtJjEiH4&index=14
- https://www.youtube.com/watch?v=NZ8FJO1LMHY&t=380s
- https://www.youtube.com/watch?v=2Z0LMc90PCM
- https://www.youtube.com/watch?v=BAk2hQgqLDk&t=96s
- https://mentecatodev.github.io/intermezzo/bots_de_telegram_en_python/
- https://www.youtube.com/watch?v=-ZyRGH-70e4&list=PL1-d6o0SXOR4_b2alX7VDkvxmfSxvKygd&index=1
- https://www.youtube.com/watch?v=YL8_aAh0v5k
- https://www.odoo.com/documentation/18.0/th/developer/reference/backend/http.html
- https://www.daniweb.com/programming/software-development/threads/488949/looking-for-random-image-generator-pil-script
- https://www.youtube.com/watch?v=bGdmZ201OXc
- https://alexwlchan.net/2022/circle-experiments/
- https://pillow.readthedocs.io/en/stable/handbook/tutorial.html
- https://pillow.readthedocs.io/en/stable/handbook/concepts.html
- https://pillow.readthedocs.io/en/stable/reference/ImageDraw.html
- https://developer.mozilla.org/en-US/docs/Web/URI/Reference/Schemes/data